

Travel Insurance Management System (Using MySQL)

Introduction

Travel insurance provides financial protection against unexpected events during travel such as medical emergencies, trip cancellation, loss of luggage, and other travel related risks.

The **Travel Insurance Management System** is a database-driven application developed using **MySQL** to manage travel insurance policies efficiently. The system stores and manages information related to customers, agents, travel plans, policies, payments, claims, and trip details.

This project helps insurance companies maintain accurate records, track policies, process claims, and manage travel plans in a structured database system.

Objective of the Project

The main objectives of this project are:

- To develop a **database system for managing travel insurance policies**.
- To store and manage **customer, agent, and travel plan information**.
- To maintain **policy details and premium payments**.
- To track **trip details and travel destinations**.
- To manage **insurance claims and claim types**.
- To ensure **efficient data storage and retrieval using SQL queries**.
- To maintain **policy status and claim processing records**.

Tools Used

Tool	Purpose
MySQL	Database Management System
MySQL Workbench / XAMPP / phpMyAdmin	Database design and query execution
SQL	Writing queries for database operations
ER Diagram Tools (optional)	Database design

Database Design

The Travel Insurance Management System database is designed using **relational database concepts**.

The system consists of multiple tables that store different types of data and are connected using **primary keys and foreign keys**.

Main entities in the database:

- Customers
- Login
- Agents
- Country
- Travel Plans
- Trip Details
- Policy
- Premium
- Payment
- Claim Type
- Claim
- Policy Status

Each table stores specific information required for managing travel insurance policies.

-- Table1 Customers

```
CREATE TABLE customers(  
customer_id INT PRIMARY KEY,  
first_name VARCHAR(50),  
last_name VARCHAR(50),gender VARCHAR(10),  
dob DATE,  
email VARCHAR(100),
```

```

phone VARCHAR(15),
address VARCHAR(200)
);

-- Table 2 Login
CREATE TABLE login(
login_id INT PRIMARY KEY,
customer_id INT,
username VARCHAR(50),
password VARCHAR(100),
FOREIGN KEY(customer_id) REFERENCES customers(customer_id)
);

-- Table 3 Agents
CREATE TABLE agents(
agent_id INT PRIMARY KEY,
agent_name VARCHAR(100),
phone VARCHAR(15),
email VARCHAR(100),
region VARCHAR(50)
);

-- Table 4 Country
CREATE TABLE country(
country_id INT PRIMARY KEY,
country_name VARCHAR(100)
);

-- Table 5 Travel plan
CREATE TABLE travel_plan(
plan_id INT PRIMARY KEY,

```

```
plan_name VARCHAR(100),
coverage_amount DECIMAL(10,2),
plan_duration INT
);
```

-- Table 6 trip details

```
CREATE TABLE trip_details(
trip_id INT PRIMARY KEY,
customer_id INT,
country_id INT,
start_date DATE,
end_date DATE,
purpose VARCHAR(100),
FOREIGN KEY(customer_id) REFERENCES customers(customer_id),
FOREIGN KEY(country_id) REFERENCES country(country_id)
);-- Table 7 Policy
```

```
CREATE TABLE policy(
policy_id INT PRIMARY KEY,
trip_id INT,
plan_id INT,
agent_id INT,
policy_status VARCHAR(20),
issue_date DATE,
expiry_date DATE,
FOREIGN KEY(trip_id) REFERENCES trip_details(trip_id),
FOREIGN KEY(plan_id) REFERENCES travel_plan(plan_id),
FOREIGN KEY(agent_id) REFERENCES agents(agent_id)
```

```
);
```

```
-- Table 8 Premium
```

```
CREATE TABLE premium(  
  premium_id INT PRIMARY KEY,  
  policy_id INT,  
  base_amount DECIMAL(10,2),  
  tax_amount DECIMAL(10,2),  
  total_amount DECIMAL(10,2),  
  FOREIGN KEY(policy_id) REFERENCES policy(policy_id)  
);
```

```
-- Table 9 Payment
```

```
CREATE TABLE payment(  
  payment_id INT PRIMARY KEY,  
  policy_id INT,  
  payment_date DATE,  
  amount_paid DECIMAL(10,2),  
  payment_method VARCHAR(50),  
  FOREIGN KEY(policy_id) REFERENCES policy(policy_id)  
);
```

```
-- Table 10 claim Type
```

```
CREATE TABLE claim_type(  
  claim_type_id INT PRIMARY KEY,  
  claim_type_name VARCHAR(100)  
);
```

```
-- table 11 claim
```

```
CREATE TABLE claim(  

```

```

claim_id INT PRIMARY KEY,
policy_id INT,
claim_type_id INT,
claim_date DATE,
claim_amount DECIMAL(10,2),claim_status VARCHAR(20),
FOREIGN KEY(policy_id) REFERENCES policy(policy_id),
FOREIGN KEY(claim_type_id) REFERENCES claim_type(claim_type_id)
);

```

-- Table 12 Policy status

```

CREATE TABLE policy_status(
status_id INT PRIMARY KEY,
status_name VARCHAR(50)
);

```

Insert values values for each table:

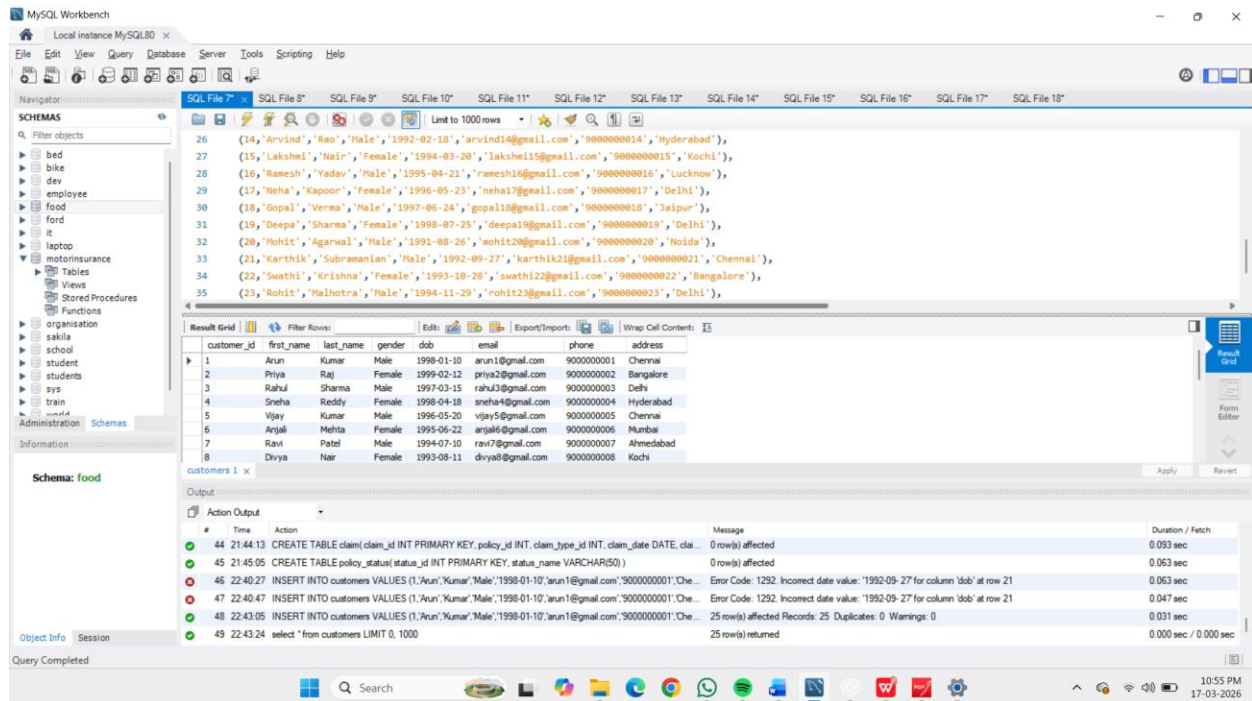
-- Insert values into customers table

```

INSERT INTO customers VALUES
(1,'Arun','Kumar','Male','1998-01-10','arun1@gmail.com','9000000001','Chennai'),
(2,'Priya','Raj','Female','1999-02-12','priya2@gmail.com','9000000002','Bangalore'),
(3,'Rahul','Sharma','Male','1997-03-15','rahul3@gmail.com','9000000003','Delhi'),
(4,'Sneha','Reddy','Female','1998-04-18','sneha4@gmail.com','9000000004','Hyderabad'),
(5,'Vijay','Kumar','Male','1996-05-20','vijay5@gmail.com','9000000005','Chennai'),
(6,'Anjali','Mehta','Female','1995-06-22','anjali6@gmail.com','9000000006','Mumbai'),
(7,'Ravi','Patel','Male','1994-07-10','ravi7@gmail.com','9000000007','Ahmedabad'),
(8,'Divya','Nair','Female','1993-08-11','divya8@gmail.com','9000000008','Kochi'),
(9,'Suresh','Babu','Male','1992-09-12','suresh9@gmail.com','9000000009','Madurai'),
(10,'Kiran','Das','Male','1991-10-10','kiran10@gmail.com','9000000010','Kolkata'),

```

(11,'Meena','Iyer','Female','1990-11-12','meena11@gmail.com','9000000011','Chennai'),
(12,'Ajay','Singh','Male','1989-12-14','ajay12@gmail.com','9000000012','Delhi'),(13,'Pooja','Gupta','Female','1993-01-16','pooja13@gmail.com','9000000013','Pune'),
(14,'Arvind','Rao','Male','1992-02-18','arvind14@gmail.com','9000000014','Hyderabad'),
(15,'Lakshmi','Nair','Female','1994-03-20','lakshmi15@gmail.com','9000000015','Kochi'),
(16,'Ramesh','Yadav','Male','1995-04-21','ramesh16@gmail.com','9000000016','Lucknow'),
(17,'Neha','Kapoor','Female','1996-05-23','neha17@gmail.com','9000000017','Delhi'),
(18,'Gopal','Verma','Male','1997-06-24','gopal18@gmail.com','9000000018','Jaipur'),
(19,'Deepa','Sharma','Female','1998-07-25','deepa19@gmail.com','9000000019','Delhi'),
(20,'Mohit','Agarwal','Male','1991-08-26','mohit20@gmail.com','9000000020','Noida'),
(21,'Karthik','Subramanian','Male','1992-09-27','karthik21@gmail.com','9000000021','Chennai'),
(22,'Swathi','Krishna','Female','1993-10-28','swathi22@gmail.com','9000000022','Bangalore'),
(23,'Rohit','Malhotra','Male','1994-11-29','rohit23@gmail.com','9000000023','Delhi'),
(24,'Anita','Desai','Female','1995-12-30','anita24@gmail.com','9000000024','Mumbai'),
(25,'Varun','Khanna','Male','1996-01-02','varun25@gmail.com','9000000025','Chandigarh');
select * from customers;



-- Insert Values into login table

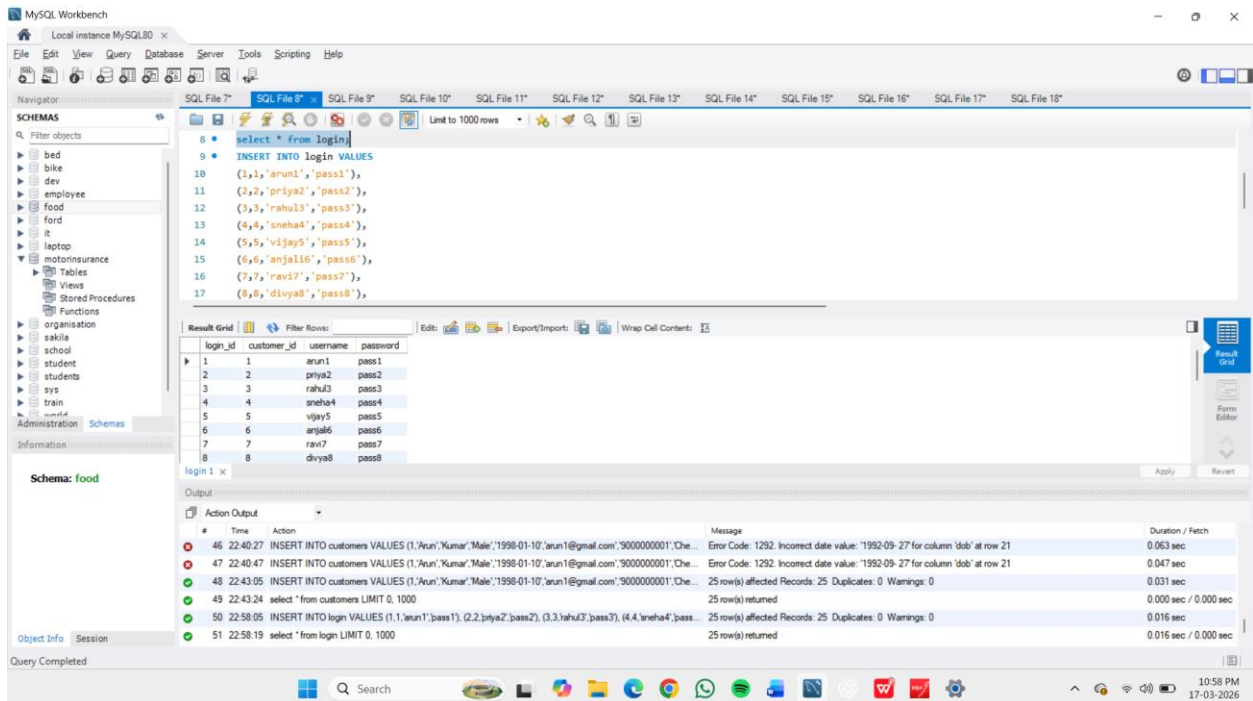
```
INSERT INTO login VALUES
(1,1,'arun1','pass1'),
(2,2,'priya2','pass2'),
(3,3,'rahul3','pass3'),
(4,4,'sneha4','pass4'),
(5,5,'vijay5','pass5'),
(6,6,'anjali6','pass6'),
(7,7,'ravi7','pass7'),
(8,8,'divya8','pass8'),
(9,9,'suresh9','pass9'),
(10,10,'kiran10','pass10'),
(11,11,'meena11','pass11'),
(12,12,'ajay12','pass12'),
(13,13,'pooja13','pass13'),
```

```

(14,14,'arvind14','pass14'),
(15,15,'lakshmi15','pass15'),
(16,16,'ramesh16','pass16'),
(17,17,'neha17','pass17'),
(18,18,'gopal18','pass18'),
(19,19,'deepa19','pass19'),
(20,20,'mohit20','pass20'),
(21,21,'karthik21','pass21'),
(22,22,'swathi22','pass22'),
(23,23,'rohit23','pass23'),
(24,24,'anita24','pass24'),
(25,25,'varun25','pass25');

select * from login;

```

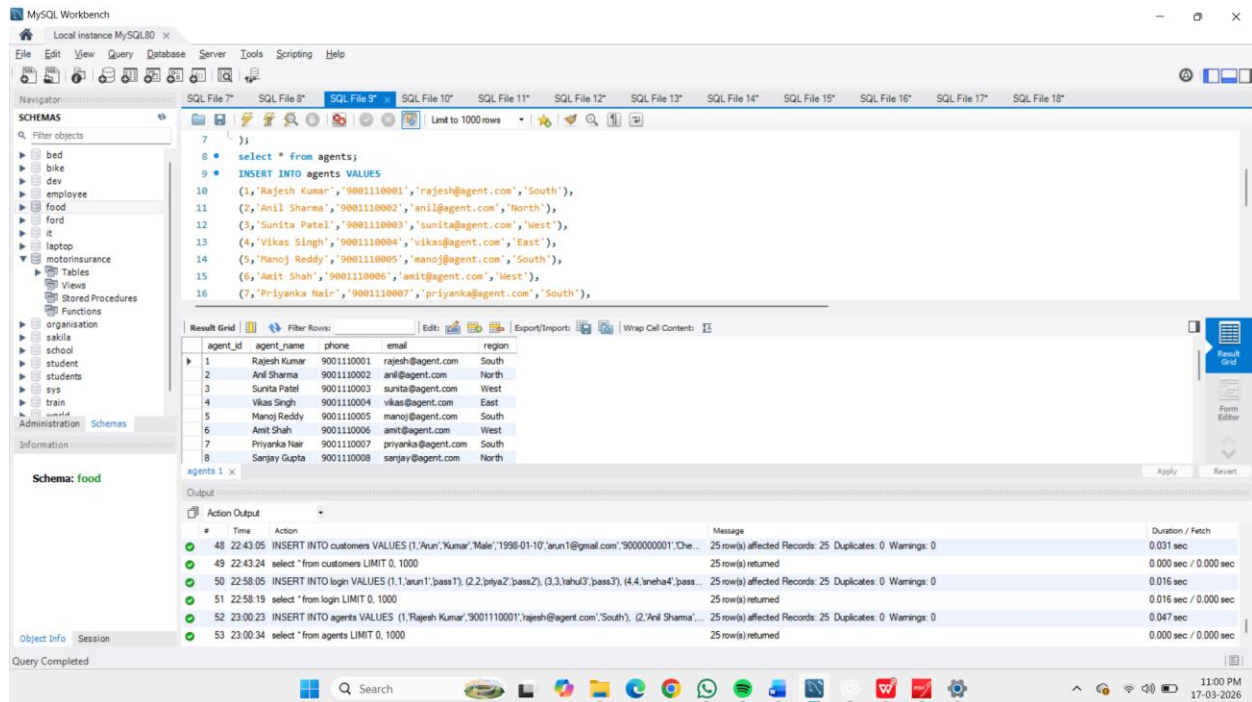


-- Insert Values into Agents

INSERT INTO agents VALUES

(1,'Rajesh Kumar','9001110001','rajesh@agent.com','South'),
(2,'Anil Sharma','9001110002','anil@agent.com','North'),
(3,'Sunita Patel','9001110003','sunita@agent.com','West'),
(4,'Vikas Singh','9001110004','vikas@agent.com','East'),
(5,'Manoj Reddy','9001110005','manoj@agent.com','South'),
(6,'Amit Shah','9001110006','amit@agent.com','West'),
(7,'Priyanka Nair','9001110007','priyanka@agent.com','South'),
(8,'Sanjay Gupta','9001110008','sanjay@agent.com','North'),
(9,'Kavita Joshi','9001110009','kavita@agent.com','West'),
(10,'Rakesh Verma','9001110010','rakesh@agent.com','North'),
(11,'Nitin Agarwal','9001110011','nitin@agent.com','East'),
(12,'Mehul Patel','9001110012','mehul@agent.com','West'),(13,'Pankaj
Yadav','9001110013','pankaj@agent.com','North'),
(14,'Arun Menon','9001110014','arun@agent.com','South'),
(15,'Deepak Jain','9001110015','deepak@agent.com','Central'),
(16,'Ritu Kapoor','9001110016','ritu@agent.com','North'),
(17,'Shalini Rao','9001110017','shalini@agent.com','South'),
(18,'Mahesh Babu','9001110018','mahesh@agent.com','South'),
(19,'Harish Kumar','9001110019','harish@agent.com','North'),
(20,'Neeraj Singh','9001110020','neeraj@agent.com','East'),
(21,'Vinay Patel','9001110021','vinay@agent.com','West'),
(22,'Siddharth Shah','9001110022','sid@agent.com','West'),
(23,'Naveen Reddy','9001110023','naveen@agent.com','South'),
(24,'Ankit Jain','9001110024','ankit@agent.com','Central'),
(25,'Rahul Mehta','9001110025','rahul@agent.com','West');

select * from agents;



-- Insert values into country table

INSERT INTO country VALUES

(1,'USA'),

(2,'UK'),

(3,'Canada'),

(4,'Australia'),

(5,'Singapore'),

(6,'Malaysia'),

(7,'Thailand'),

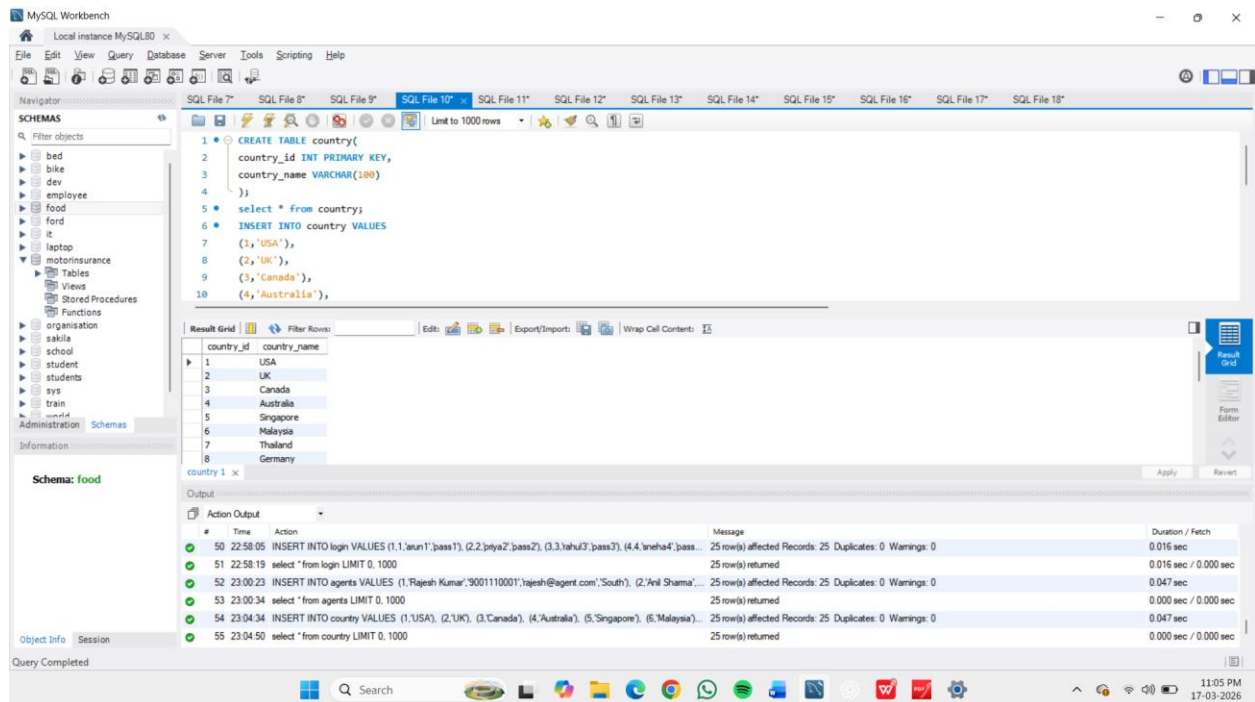
(8,'Germany'),

(9,'France'),

(10,'Italy'),

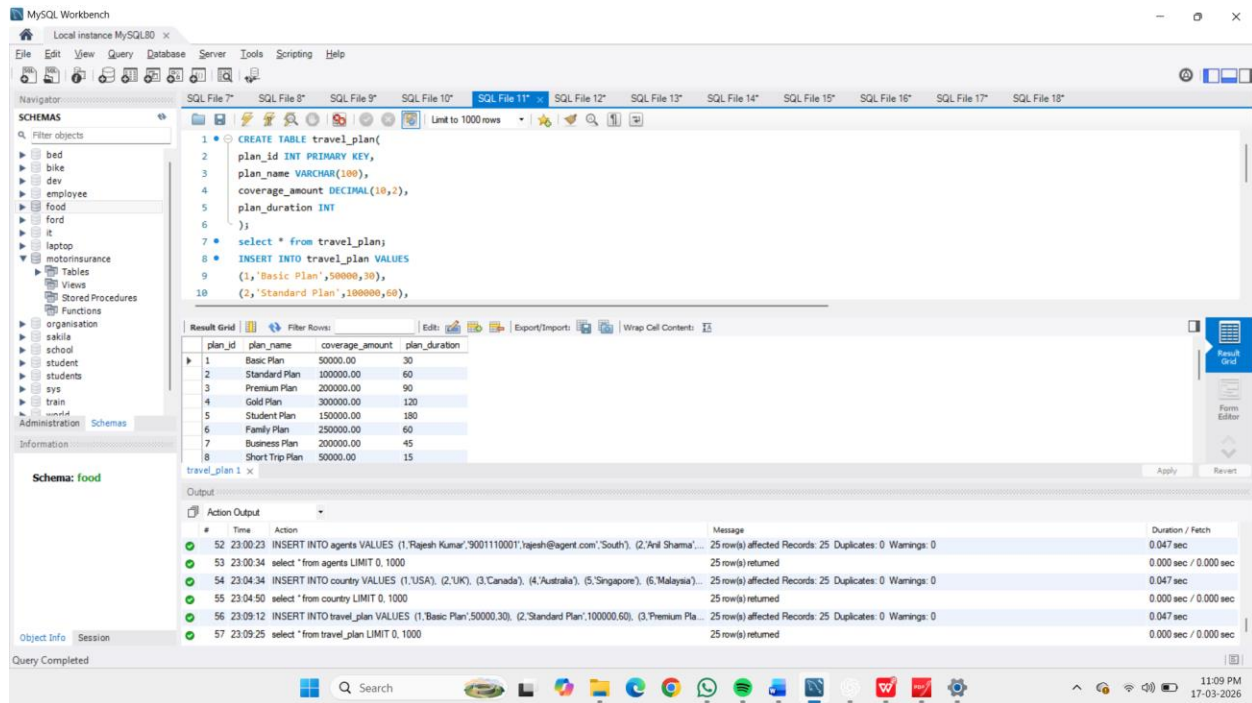
(11,'Spain'),

(12,'Japan'),
 (13,'South Korea'),
 (14,'China'),
 (15,'UAE'),
 (16,'Qatar'),
 (17,'Saudi Arabia'),
 (18,'New Zealand'),
 (19,'Netherlands'),
 (20,'Switzerland'),
 (21,'Sweden'),
 (22,'Norway'),
 (23,'Denmark'),
 (24,'Finland'),
 (25,'Brazil');
 select * from country;



-- Insert Values into Travel Plan Table

```
INSERT INTO travel_plan VALUES
(1,'Basic Plan',50000,30),
(2,'Standard Plan',100000,60),
(3,'Premium Plan',200000,90),
(4,'Gold Plan',300000,120),
(5,'Student Plan',150000,180),
(6,'Family Plan',250000,60),
(7,'Business Plan',200000,45),
(8,'Short Trip Plan',50000,15),
(9,'Holiday Plan',120000,30),(10,'Corporate Plan',300000,90),
(11,'Explorer Plan',180000,60),
(12,'Adventure Plan',200000,30),
(13,'Asia Plan',150000,45),
(14,'Europe Plan',250000,60),
(15,'World Plan',500000,90),
(16,'Silver Plan',80000,30),
(17,'Diamond Plan',400000,120),
(18,'Budget Plan',40000,20),
(19,'Couple Plan',120000,30),
(20,'Senior Plan',100000,45),
(21,'Student Premium',220000,180),
(22,'Frequent Traveler',350000,365),
(23,'Luxury Plan',600000,90),
(24,'Backpacker Plan',70000,60),
(25,'Elite Plan',700000,180);
select * from travel_plan;
```



-- Insert Values into trip details table

INSERT INTO trip_details VALUES

(1,1,1,'2026-01-01','2026-01-10','Business'),

(2,2,2,'2026-01-05','2026-01-15','Vacation'),

(3,3,3,'2026-01-10','2026-01-20','Study'),

(4,4,4,'2026-01-12','2026-01-25','Tourism'),

(5,5,5,'2026-01-15','2026-01-28','Business'),

(6,6,6,'2026-01-18','2026-02-01','Vacation'),

(7,7,7,'2026-01-20','2026-02-02','Tourism'),

(8,8,8,'2026-01-22','2026-02-05','Study'),

(9,9,9,'2026-01-25','2026-02-07','Business'),

(10,10,10,'2026-01-27','2026-02-10','Vacation'),

(11,11,11,'2026-02-01','2026-02-15','Tourism'),

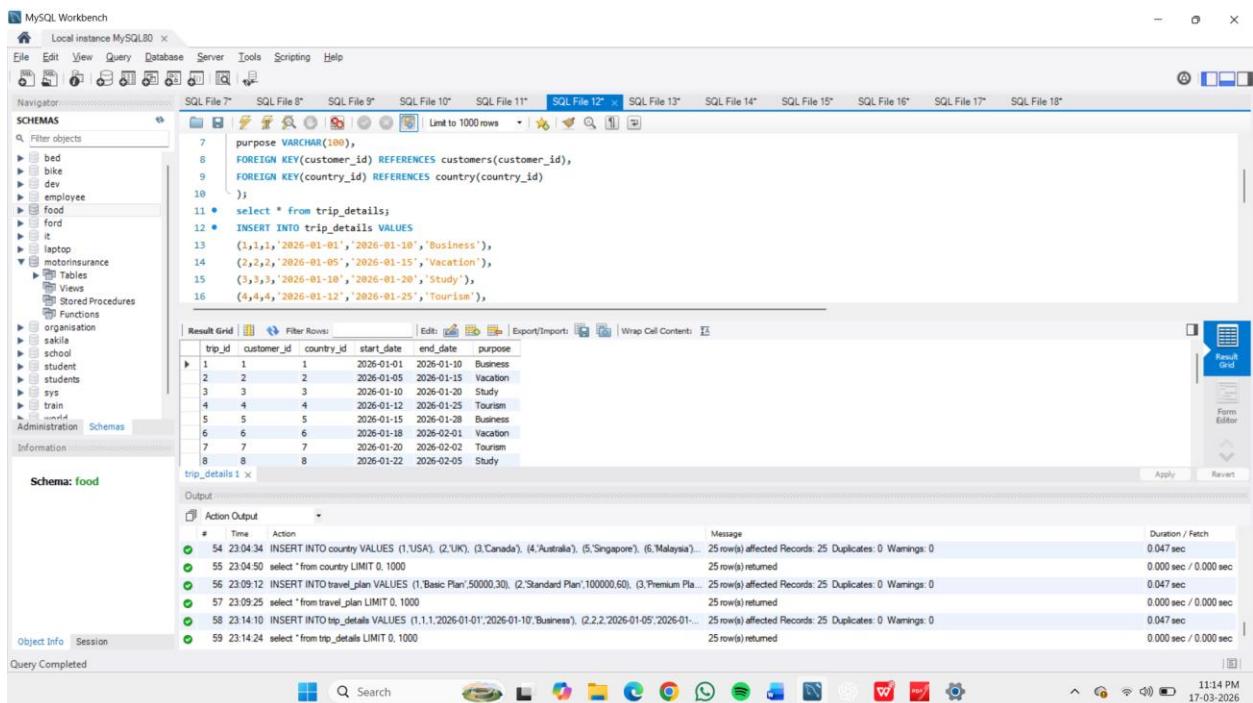
(12,12,12,'2026-02-03','2026-02-18','Study'),

```

(13,13,13,'2026-02-05','2026-02-20','Business'),
(14,14,14,'2026-02-07','2026-02-22','Vacation'),
(15,15,15,'2026-02-10','2026-02-25','Tourism'),
(16,16,16,'2026-02-12','2026-02-28','Study'),
(17,17,17,'2026-02-14','2026-03-01','Business'),
(18,18,18,'2026-02-16','2026-03-05','Vacation'),
(19,19,19,'2026-02-18','2026-03-08','Tourism'),
(20,20,20,'2026-02-20','2026-03-10','Study'),
(21,21,21,'2026-02-22','2026-03-12','Business'),
(22,22,22,'2026-02-24','2026-03-15','Vacation'),
(23,23,23,'2026-02-26','2026-03-18','Tourism'),
(24,24,24,'2026-02-28','2026-03-20','Study'),
(25,25,25,'2026-03-01','2026-03-22','Business');

select * from trip_details;

```



-- Insert Values into policy table

INSERT INTO policy VALUES

(1,1,1,1,'Active','2026-01-01','2026-01-30'),
(2,2,2,2,'Active','2026-01-05','2026-03-05'),
(3,3,3,3,'Active','2026-01-10','2026-04-10'),
(4,4,4,4,'Active','2026-01-12','2026-05-12'),
(5,5,5,5,'Active','2026-01-15','2026-07-15'),
(6,6,6,6,'Active','2026-01-18','2026-03-18'),
(7,7,7,7,'Active','2026-01-20','2026-03-20'),
(8,8,8,8,'Active','2026-01-22','2026-02-22'),
(9,9,9,9,'Active','2026-01-25','2026-02-25'),
(10,10,10,10,'Active','2026-01-27','2026-04-27'),
(11,11,11,11,'Active','2026-02-01','2026-04-01'),
(12,12,12,12,'Active','2026-02-03','2026-04-03'),
(13,13,13,13,'Active','2026-02-05','2026-04-05'),
(14,14,14,14,'Active','2026-02-07','2026-04-07'),
(15,15,15,15,'Active','2026-02-10','2026-04-10'),
(16,16,16,16,'Active','2026-02-12','2026-04-12'),
(17,17,17,17,'Active','2026-02-14','2026-04-14'),
(18,18,18,18,'Active','2026-02-16','2026-04-16'),
(19,19,19,19,'Active','2026-02-18','2026-04-18'),
(20,20,20,20,'Active','2026-02-20','2026-04-20'),
(21,21,21,21,'Active','2026-02-22','2026-04-22'),
(22,22,22,22,'Active','2026-02-24','2026-04-24'),
(23,23,23,23,'Active','2026-02-26','2026-04-26'),
(24,24,24,24,'Active','2026-02-28','2026-04-28'),
(25,25,25,25,'Active','2026-03-01','2026-05-01');

select * from policy;

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
5 agent_id INT,  
6 policy_status VARCHAR(20),  
7 issue_date DATE,  
8 expiry_date DATE,  
9 FOREIGN KEY(trip_id) REFERENCES trip_details(trip_id),  
10 FOREIGN KEY(plan_id) REFERENCES travel_plan(plan_id),  
11 FOREIGN KEY(agent_id) REFERENCES agents(agent_id)  
12 );  
13 select * from policy;  
14 INSERT INTO policy VALUES
```

The Result Grid shows the following data:

policy_id	trip_id	plan_id	agent_id	policy_status	issue_date	expiry_date
1	1	1	1	Active	2026-01-01	2026-01-30
2	2	2	2	Active	2026-01-05	2026-03-05
3	3	3	3	Active	2026-01-10	2026-04-10
4	4	4	4	Active	2026-01-12	2026-05-12
5	5	5	5	Active	2026-01-15	2026-07-15
6	6	6	6	Active	2026-01-18	2026-03-18
7	7	7	7	Active	2026-01-20	2026-03-20
8	8	8	8	Active	2026-01-22	2026-02-22

The Action Output shows the following results:

#	Time	Action	Message	Duration / Fetch
56	23:09:12	INSERT INTO travel_plan VALUES (1,'Basic Plan',50000,30), (2,'Standard Plan',100000,60), (3,'Premium Plan',150000,90);	25 row(s) affected Records: 25 Duplicates: 0 Warnings: 0	0.047 sec
57	23:09:25	select * from travel_plan LIMIT 0, 1000	25 row(s) returned	0.000 sec / 0.000 sec
58	23:14:10	INSERT INTO trip_details VALUES (1,1,1,'2026-01-01','2026-01-10','Business'), (2,2,2,'2026-01-05','2026-01-15','Business'), (3,3,3,'2026-01-10','2026-01-20','Business'), (4,4,4,'2026-01-12','2026-01-22','Business'), (5,5,5,'2026-01-15','2026-01-25','Business'), (6,6,6,'2026-01-18','2026-01-28','Business'), (7,7,7,'2026-01-20','2026-01-30','Business'), (8,8,8,'2026-01-22','2026-02-02','Business');	25 row(s) affected Records: 25 Duplicates: 0 Warnings: 0	0.047 sec
59	23:14:24	select * from trip_details LIMIT 0, 1000	25 row(s) returned	0.000 sec / 0.000 sec
60	23:16:48	INSERT INTO policy VALUES (1,1,1,1,'Active','2026-01-01','2026-01-30'), (2,2,2,2,'Active','2026-01-05','2026-01-30'), (3,3,3,3,'Active','2026-01-10','2026-01-30'), (4,4,4,4,'Active','2026-01-12','2026-01-30'), (5,5,5,5,'Active','2026-01-15','2026-01-30'), (6,6,6,6,'Active','2026-01-18','2026-01-30'), (7,7,7,7,'Active','2026-01-20','2026-01-30'), (8,8,8,8,'Active','2026-01-22','2026-01-30');	25 row(s) affected Records: 25 Duplicates: 0 Warnings: 0	0.031 sec
61	23:16:58	select * from policy LIMIT 0, 1000	25 row(s) returned	0.000 sec / 0.000 sec

-- Insert Values into premium

INSERT INTO premium VALUES

(1,1,5000,900,5900),

(2,2,8000,1440,9440),

(3,3,10000,1800,11800),

(4,4,12000,2160,14160),

(5,5,15000,2700,17700),

(6,6,7000,1260,8260),

(7,7,6000,1080,7080),

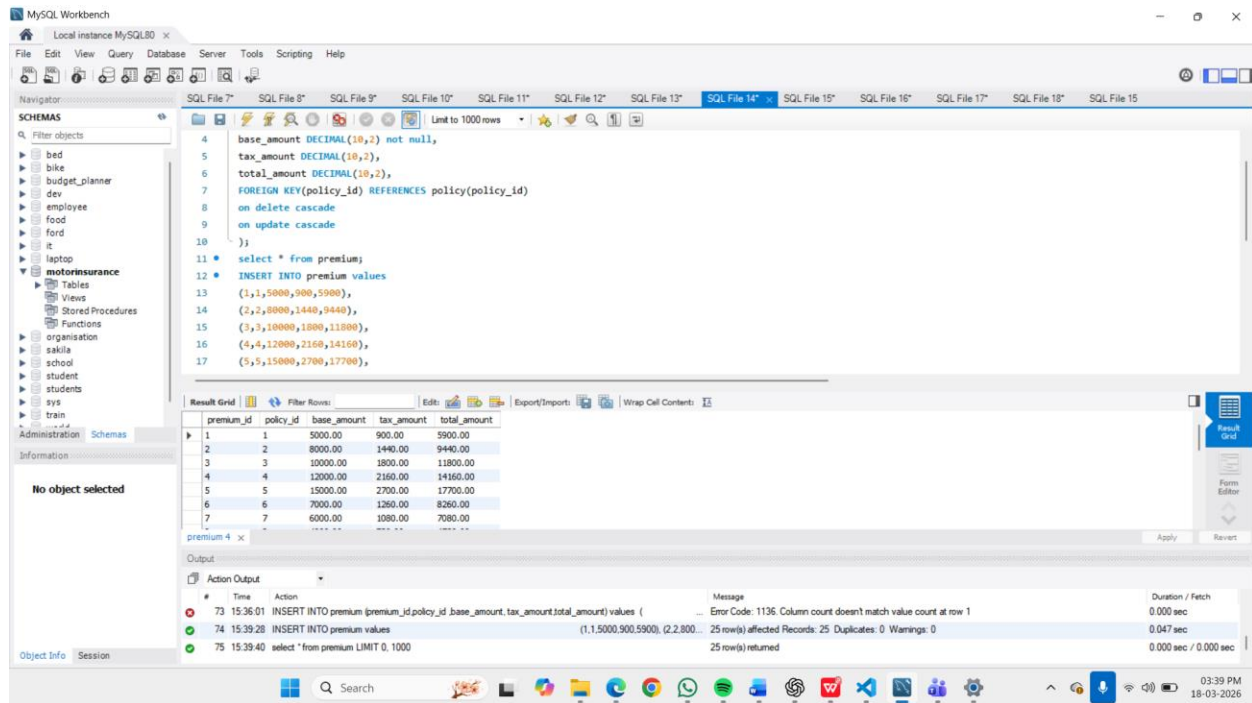
(8,8,4000,720,4720),

(9,9,4500,810,5310),

(10,10,9000,1620,10620),

(11,11,7500,1350,8850),

(12,12,7800,1404,9204),
(13,13,8200,1476,9676),
(14,14,8500,1530,10030),
(15,15,8800,1584,10384),
(16,16,9100,1638,10738),
(17,17,9400,1692,11092),
(18,18,9700,1746,11446),
(19,19,10000,1800,11800),
(20,20,10300,1854,12154),
(21,21,10600,1908,12508),
(22,22,10900,1962,12862),
(23,23,11200,2016,13216),
(24,24,11500,2070,13570),
(25,25,11800,2124,13924);
select * from premium;



-- insert Values into Payments

INSERT INTO payment VALUES

(1,1,'2026-01-01',5900,'UPI'),

(2,2,'2026-01-05',9440,'Credit Card'),

(3,3,'2026-01-10',11800,'Debit Card'),

(4,4,'2026-01-12',14160,'Net Banking'),

(5,5,'2026-01-15',17700,'UPI'),

(6,6,'2026-01-18',8260,'UPI'),

(7,7,'2026-01-20',7080,'Credit Card'),

(8,8,'2026-01-22',4720,'Debit Card'),

(9,9,'2026-01-25',5310,'UPI'),

(10,10,'2026-01-27',10620,'Net Banking'),

(11,11,'2026-02-01',8850,'UPI'),(12,12,'2026-02-03',9204,'Credit Card'),

(13,13,'2026-02-05',9676,'Debit Card'),

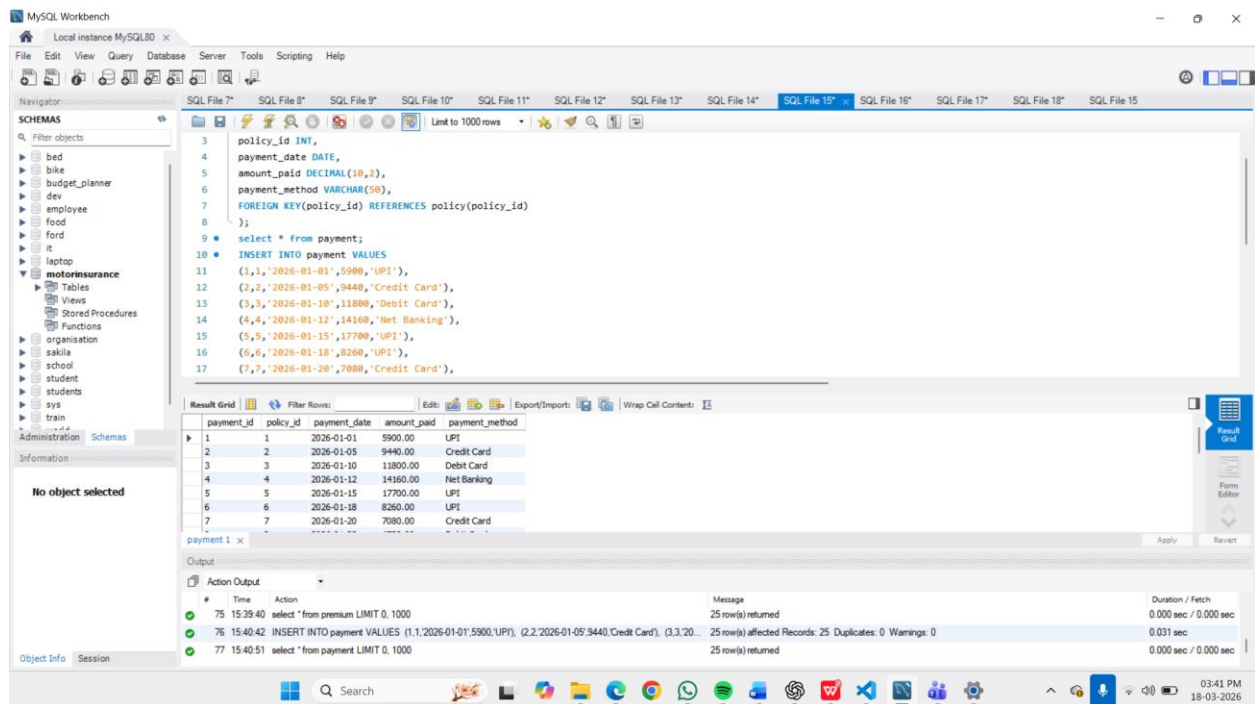
(14,14,'2026-02-07',10030,'UPI'),

```

(15,15,'2026-02-10',10384,'Net Banking'),
(16,16,'2026-02-12',10738,'UPI'),
(17,17,'2026-02-14',11092,'Credit Card'),
(18,18,'2026-02-16',11446,'Debit Card'),
(19,19,'2026-02-18',11800,'UPI'),
(20,20,'2026-02-20',12154,'Net Banking'),
(21,21,'2026-02-22',12508,'UPI'),
(22,22,'2026-02-24',12862,'Credit Card'),
(23,23,'2026-02-26',13216,'Debit Card'),
(24,24,'2026-02-28',13570,'UPI'),
(25,25,'2026-03-01',13924,'Net Banking');

select * from payment;

```



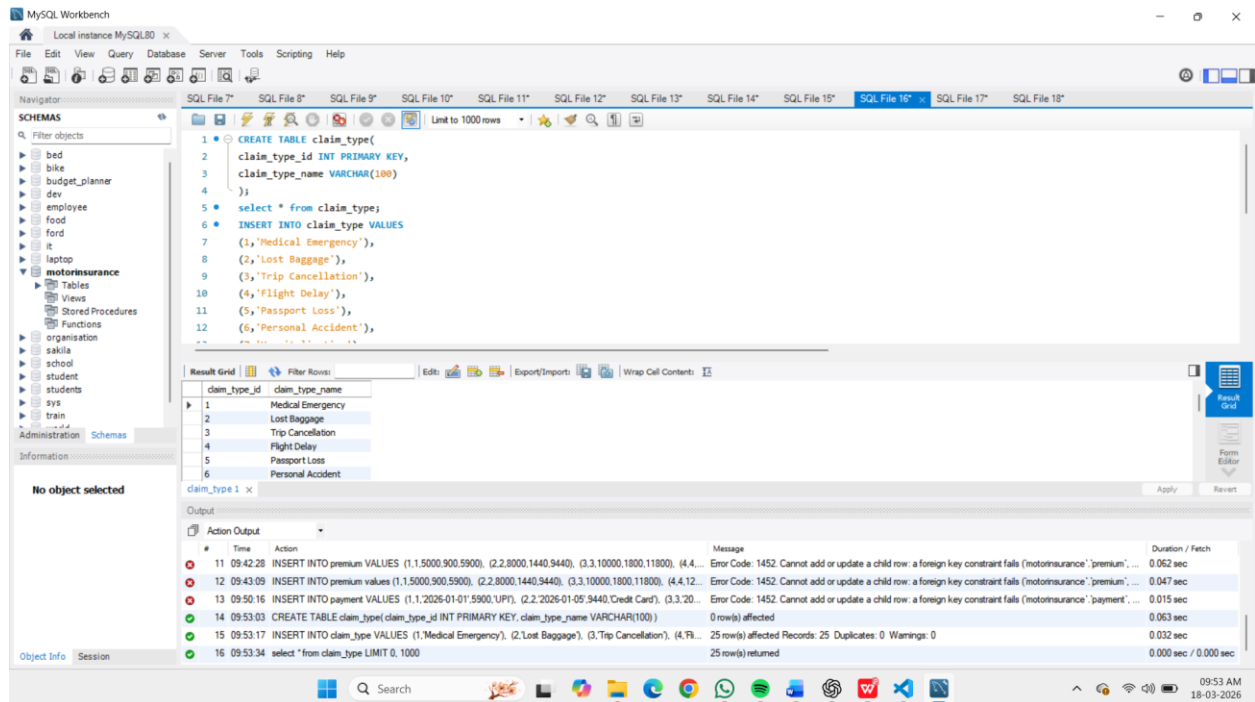
-- Insert Values into claim_type

INSERT INTO claim_type VALUES

(1,'Medical Emergency'),

(2,'Lost Baggage'),
(3,'Trip Cancellation'),
(4,'Flight Delay'),
(5,'Passport Loss'),
(6,'Personal Accident'),
(7,'Hospitalization'),
(8,'Travel Delay'),
(9,'Emergency Evacuation'),
(10,'Trip Curtailment'),
(11,'Baggage Delay'),
(12,'Hotel Cancellation'),
(13,'Travel Document Loss'),
(14,'Natural Disaster'),
(15,'Theft'),
(16,'Injury'),
(17,'Vehicle Accident'),
(18,'Emergency Treatment'),
(19,'Flight Cancellation'),
(20,'Missed Connection'),
(21,'Weather Delay'),
(22,'Emergency Return'),
(23,'Lost Money'),
(24,'Medical Transport'),
(25,'Other');

select * from claim_type;



-- Insert Values into Claim table

INSERT INTO claim VALUES

(1,1,1,'2026-01-05',2000,'Approved'),

(2,2,2,'2026-01-10',1500,'Pending'),

(3,3,3,'2026-01-12',3000,'Approved'),

(4,4,4,'2026-01-15',1200,'Rejected'),

(5,5,5,'2026-01-18',2500,'Approved'),

(6,6,6,'2026-01-20',1800,'Pending'),

(7,7,7,'2026-01-22',2200,'Approved'),

(8,8,8,'2026-01-25',1400,'Rejected'),

(9,9,9,'2026-01-27',2700,'Approved'),(10,10,10,'2026-01-30',3100,'Pending'),

(11,11,11,'2026-02-02',1900,'Approved'),

(12,12,12,'2026-02-05',2100,'Pending'),

(13,13,13,'2026-02-07',2600,'Approved'),

```
(14,14,14,'2026-02-10',2800,'Rejected'),  
(15,15,15,'2026-02-12',3200,'Approved'),  
(16,16,16,'2026-02-14',2400,'Pending'),  
(17,17,17,'2026-02-16',2900,'Approved'),  
(18,18,18,'2026-02-18',1700,'Rejected'),  
(19,19,19,'2026-02-20',3300,'Approved'),  
(20,20,20,'2026-02-22',3500,'Pending'),  
(21,21,21,'2026-02-24',2100,'Approved'),  
(22,22,22,'2026-02-26',2300,'Rejected'),  
(23,23,23,'2026-02-28',2600,'Approved'),  
(24,24,24,'2026-03-01',2800,'Pending'),  
(25,25,25,'2026-03-03',3000,'Approved');  
  
select * from claim;
```

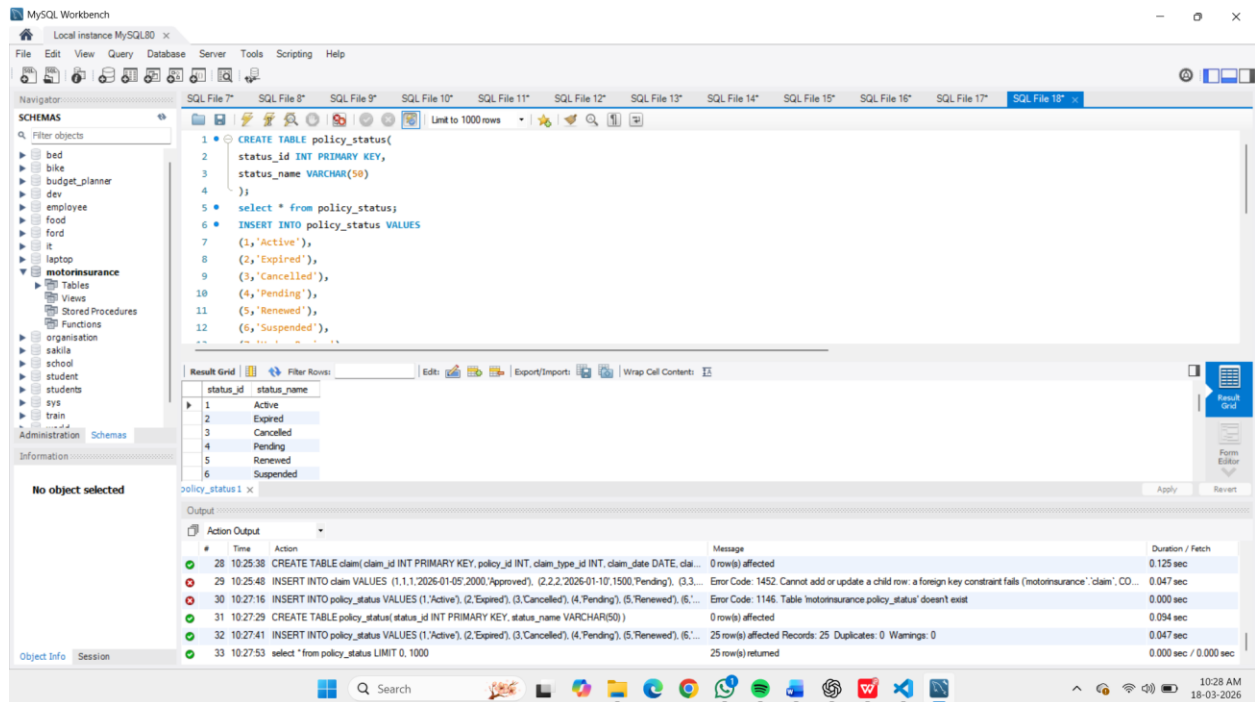
-- insert values into policy_status

```
INSERT INTO policy_status VALUES
```

```
(1,'Active'),  
(2,'Expired'),  
(3,'Cancelled'),  
(4,'Pending'),  
(5,'Renewed'),  
(6,'Suspended'),  
(7,'Under Review'),  
(8,'Approved'),  
(9,'Rejected'),
```

(10,'Processing'),
(11,'Issued'),
(12,'Completed'),
(13,'Closed'),
(14,'Reactivated'),
(15,'In Progress'),
(16,'Awaiting Payment'),
(17,'Verified'),
(18,'Terminated'),
(19,'Paused'),
(20,'Inactive'),
(21,'Confirmed'),
(22,'Draft'),
(23,'Pending Approval'),
(24,'On Hold'),
(25,'Archived');

select * from policy_status;



SQL Query Implementation:

1. DDL Commands

Create Table

```
CREATE TABLE test_table (  
    id INT PRIMARY KEY,  
    name VARCHAR(50)  
);
```

Alter Table

ALTER TABLE customers

ADD COLUMN nationality VARCHAR(50);

Drop Table

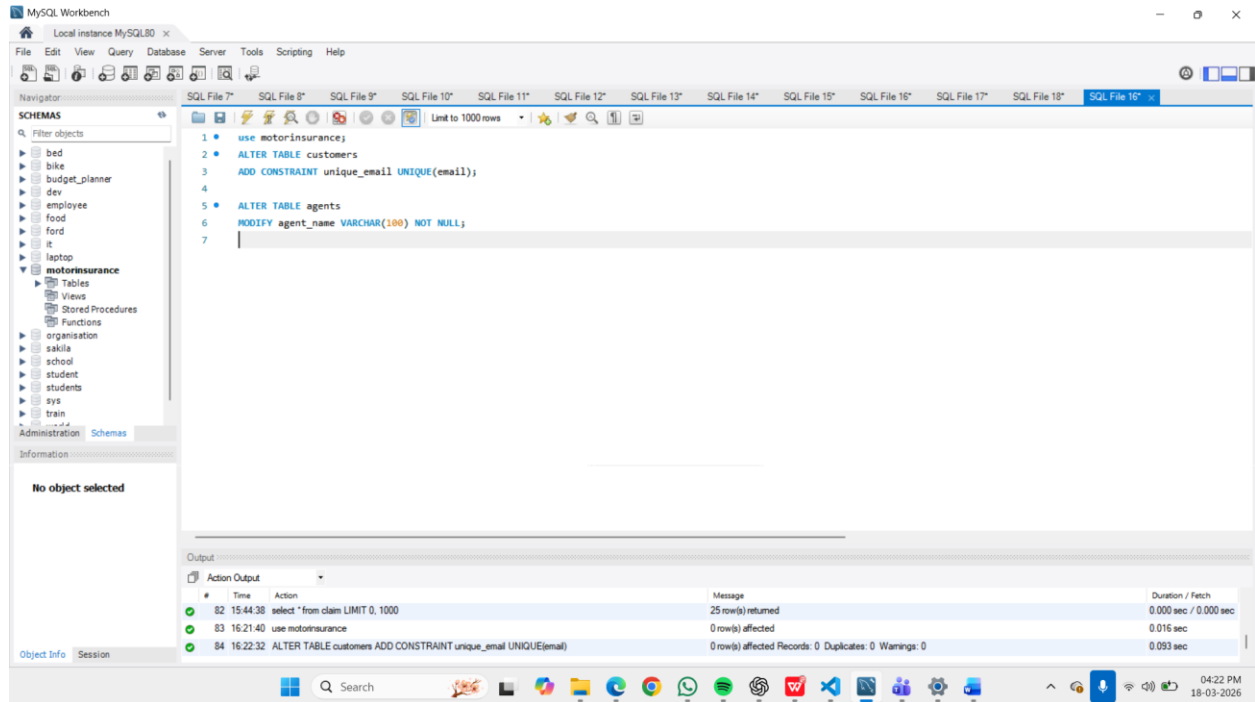
DROP TABLE test_table;

2. Constraints

Add UNIQUE Constraint

ALTER TABLE customers

ADD CONSTRAINT unique_email UNIQUE(email);



Add NOT NULL

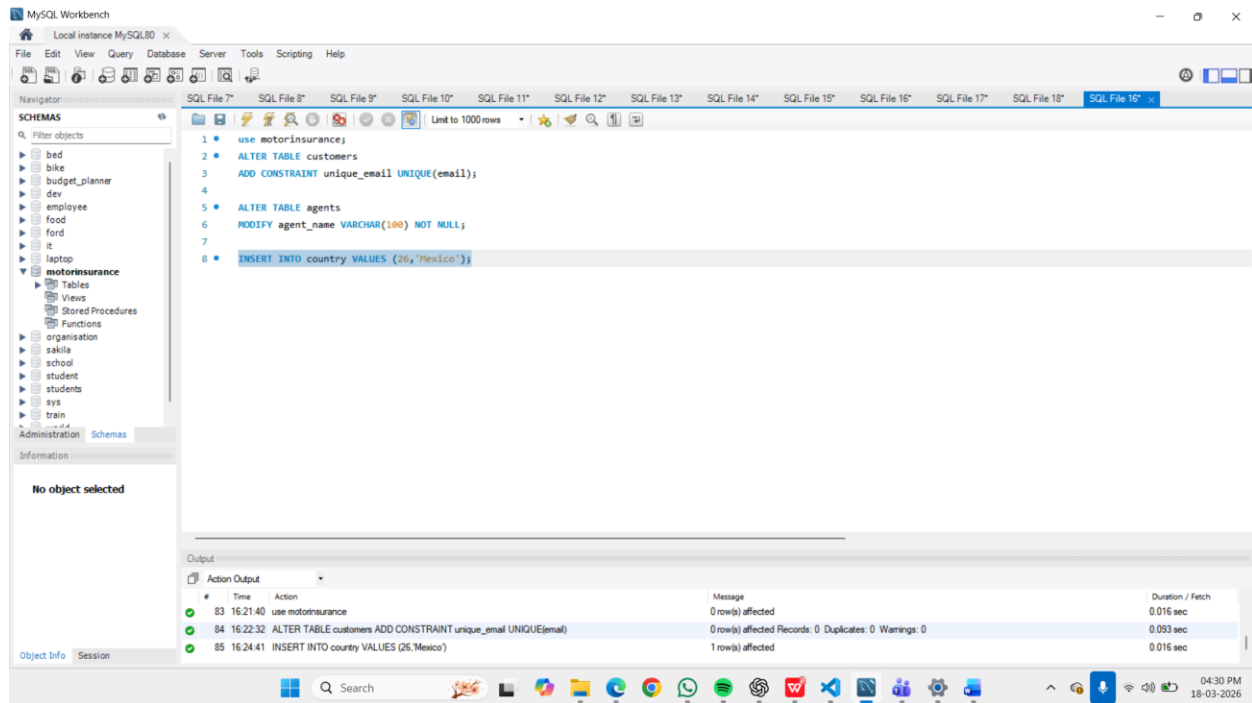
ALTER TABLE agents

MODIFY agent_name VARCHAR(100) NOT NULL;

3. DML Commands

Insert

INSERT INTO country VALUES (26,'Mexico');

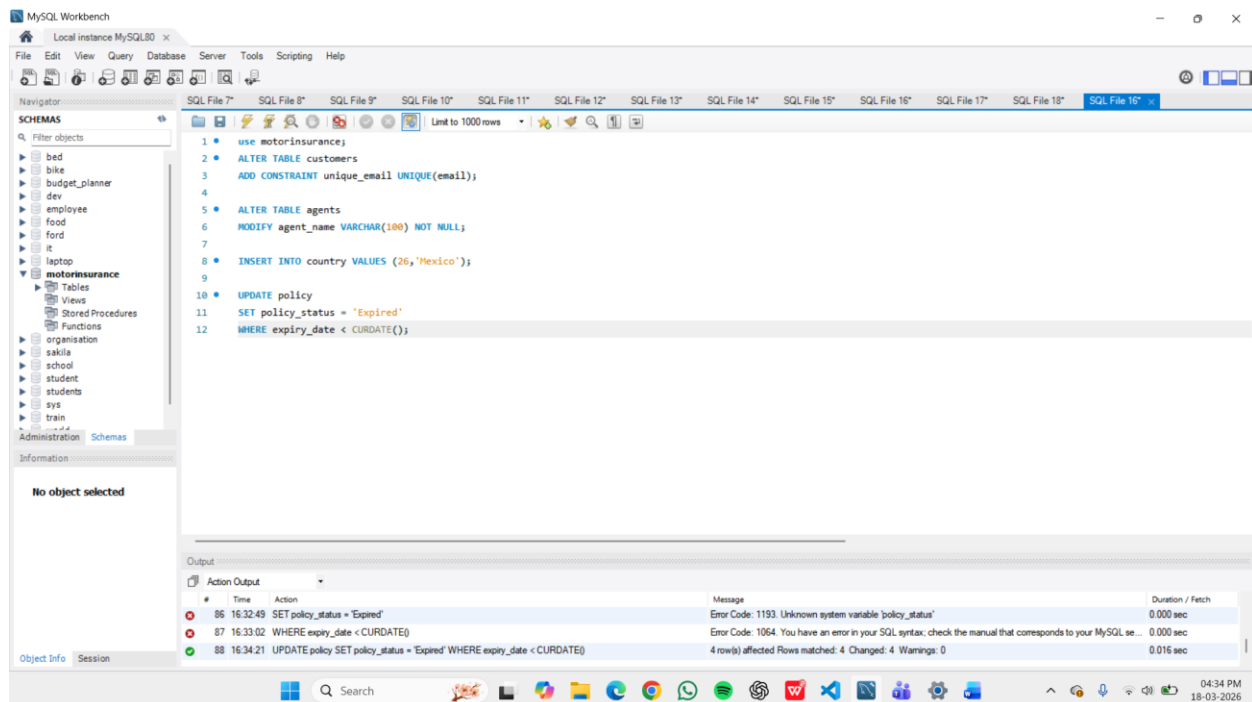


Update

UPDATE policy

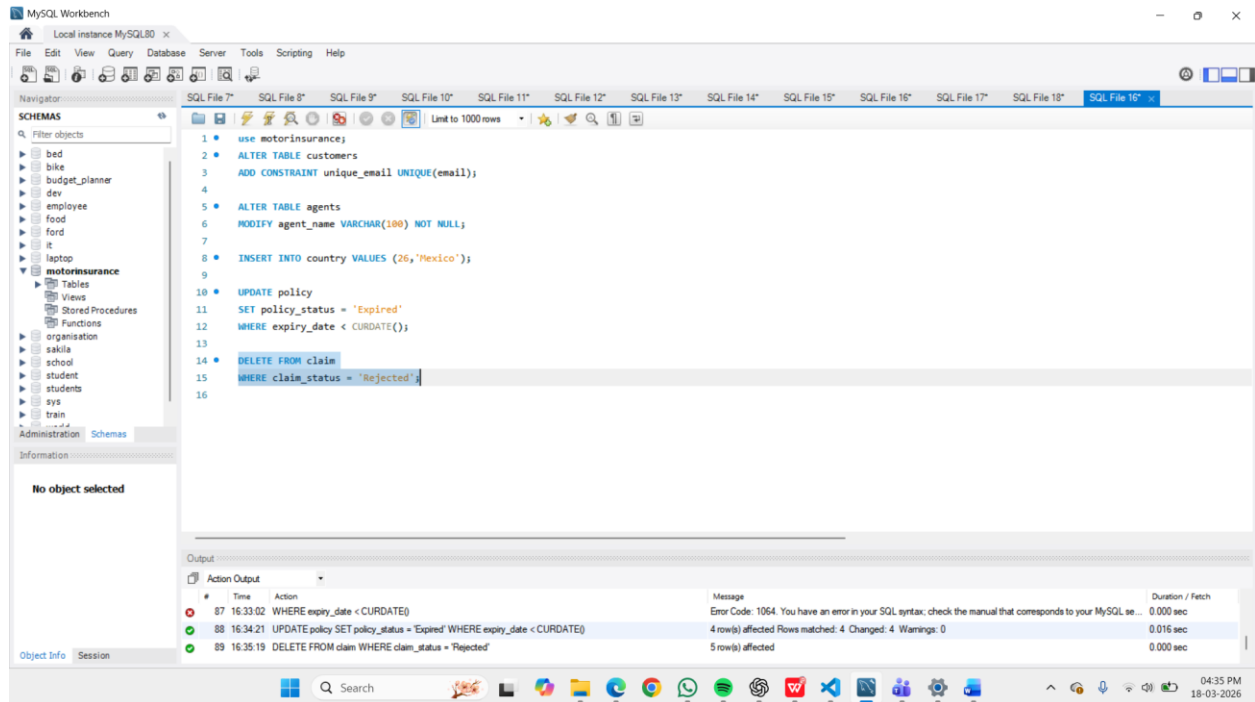
SET policy_status = 'Expired'

WHERE expiry_date < CURDATE();



Delete

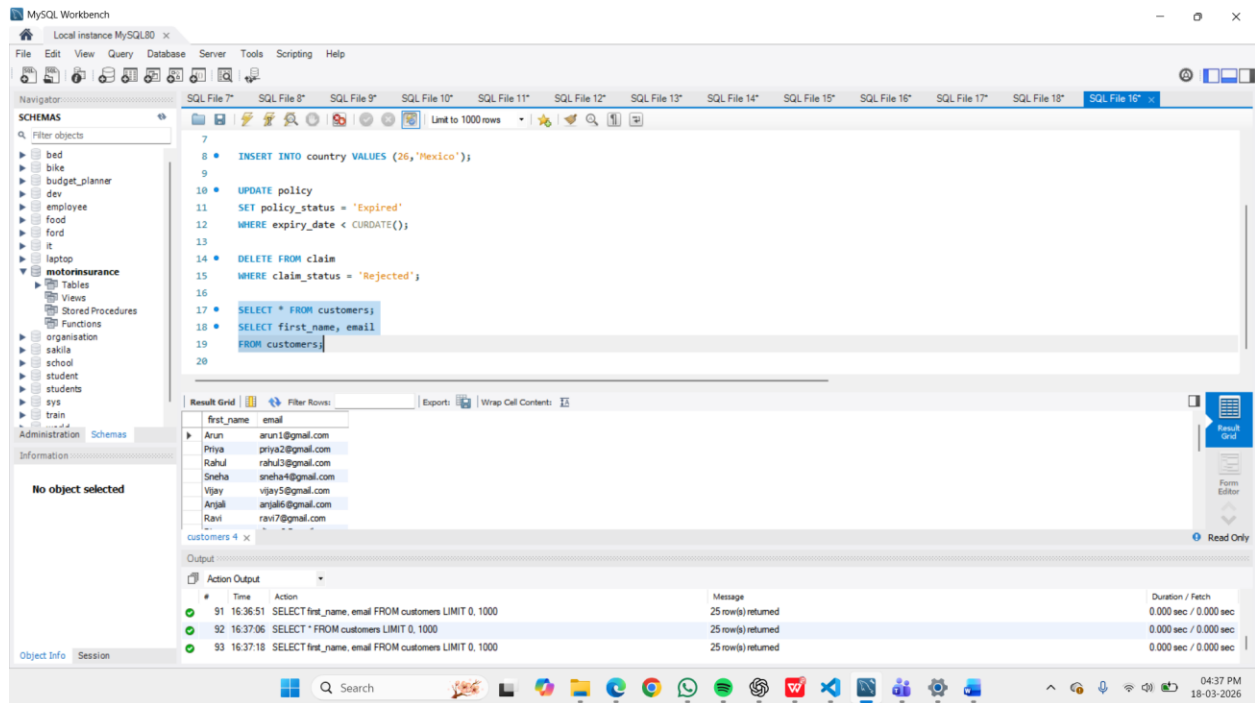
DELETE FROM claim
WHERE claim_status = 'Rejected';



4. Basic SELECT Queries

SELECT * FROM customers;

SELECT first_name, email
FROM customers;



5. Aggregation & Grouping

Total Premium Collection

SELECT SUM(total_amount) AS total_collection
FROM premium;

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

Filter objects

- bed
- bike
- budget_planner
- dev
- employee
- food
- ford
- it
- laptop
- motorinsurance
 - Tables
 - Views
 - Stored Procedures
 - Functions
- organisation
- sakila
- school
- student
- students
- sys
- train

Administration Schemas

Information

No object selected

Object Info Session

SQL File 7* SQL File 8* SQL File 9* SQL File 10* SQL File 11* SQL File 12* SQL File 13* SQL File 14* SQL File 15* SQL File 16* SQL File 17* SQL File 18* SQL File 16*

Limit to 1000 rows

```

9
10 UPDATE policy
11 SET policy_status = 'Expired'
12 WHERE expiry_date < CURDATE();
13
14 DELETE FROM claim
15 WHERE claim_status = 'Rejected';
16
17 SELECT * FROM customers;
18 SELECT first_name, email
19 FROM customers;
20
21 SELECT SUM(total_amount) AS total_collection
22 FROM premium;

```

Result Grid

total_collection

266444.00

Result 5 x

Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
92	16:37:06	SELECT * FROM customers LIMIT 0, 1000	25 row(s) returned	0.000 sec / 0.000 sec
93	16:37:18	SELECT first_name, email FROM customers LIMIT 0, 1000	25 row(s) returned	0.000 sec / 0.000 sec
94	16:38:43	SELECT SUM(total_amount) AS total_collection FROM premium LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec

04:38 PM
18-03-2026

Average Claim Amount

SELECT AVG(claim_amount) AS avg_claim
FROM claim;

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

Filter objects

- bed
- bike
- budget_planner
- dev
- employee
- food
- ford
- it
- laptop
- motorinsurance
 - Tables
 - Views
 - Stored Procedures
 - Functions
- organisation
- sakila
- school
- student
- students
- sys
- train

Administration Schemas

Information

No object selected

Object Info Session

SQL File 7* SQL File 8* SQL File 9* SQL File 10* SQL File 11* SQL File 12* SQL File 13* SQL File 14* SQL File 15* SQL File 16* SQL File 17* SQL File 18* SQL File 16*

Limit to 1000 rows

```

12 WHERE expiry_date < CURDATE();
13 Execute the selected portion of the script or everything, if there is no selection
14 DELETE FROM claim
15 WHERE claim_status = 'Rejected';
16
17 SELECT * FROM customers;
18 SELECT first_name, email
19 FROM customers;
20
21 SELECT SUM(total_amount) AS total_collection
22 FROM premium;
23
24 SELECT AVG(claim_amount) AS avg_claim
25 FROM claim;

```

Result Grid

avg_claim

2560.000000

Result 6 x

Read Only

Output

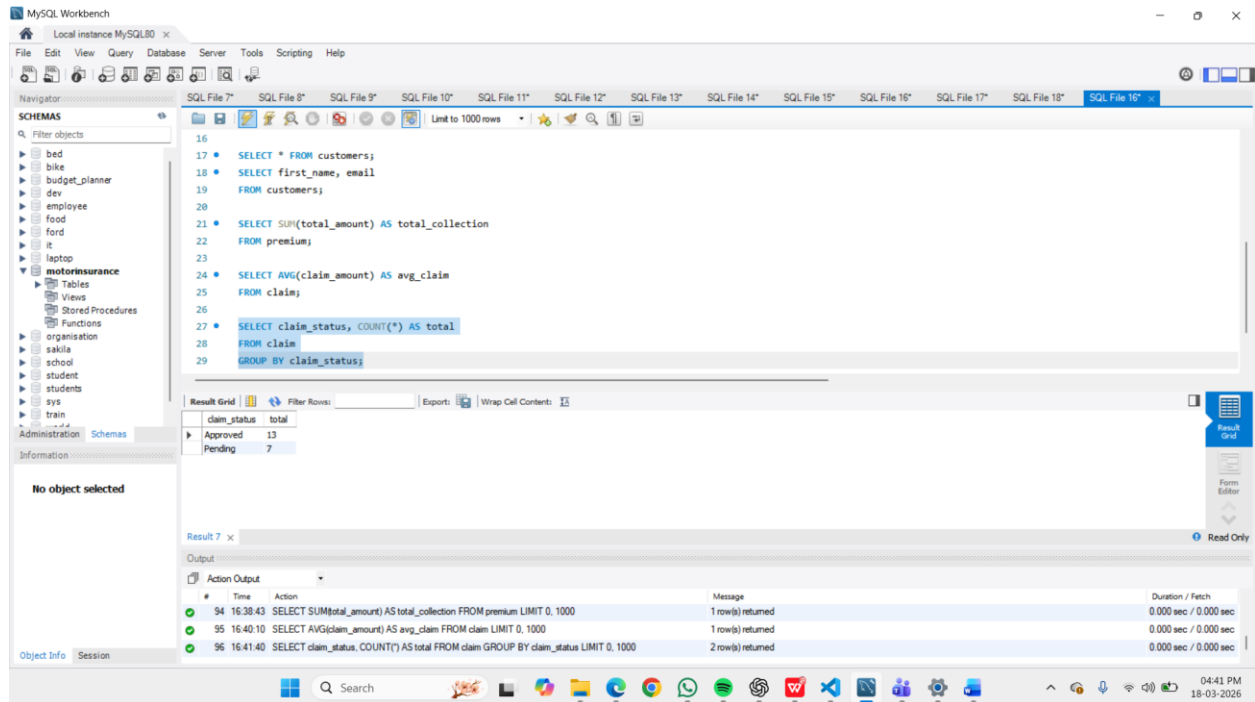
Action Output

#	Time	Action	Message	Duration / Fetch
93	16:37:18	SELECT first_name, email FROM customers LIMIT 0, 1000	25 row(s) returned	0.000 sec / 0.000 sec
94	16:38:43	SELECT SUM(total_amount) AS total_collection FROM premium LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
95	16:40:10	SELECT AVG(claim_amount) AS avg_claim FROM claim LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec

04:40 PM
18-03-2026

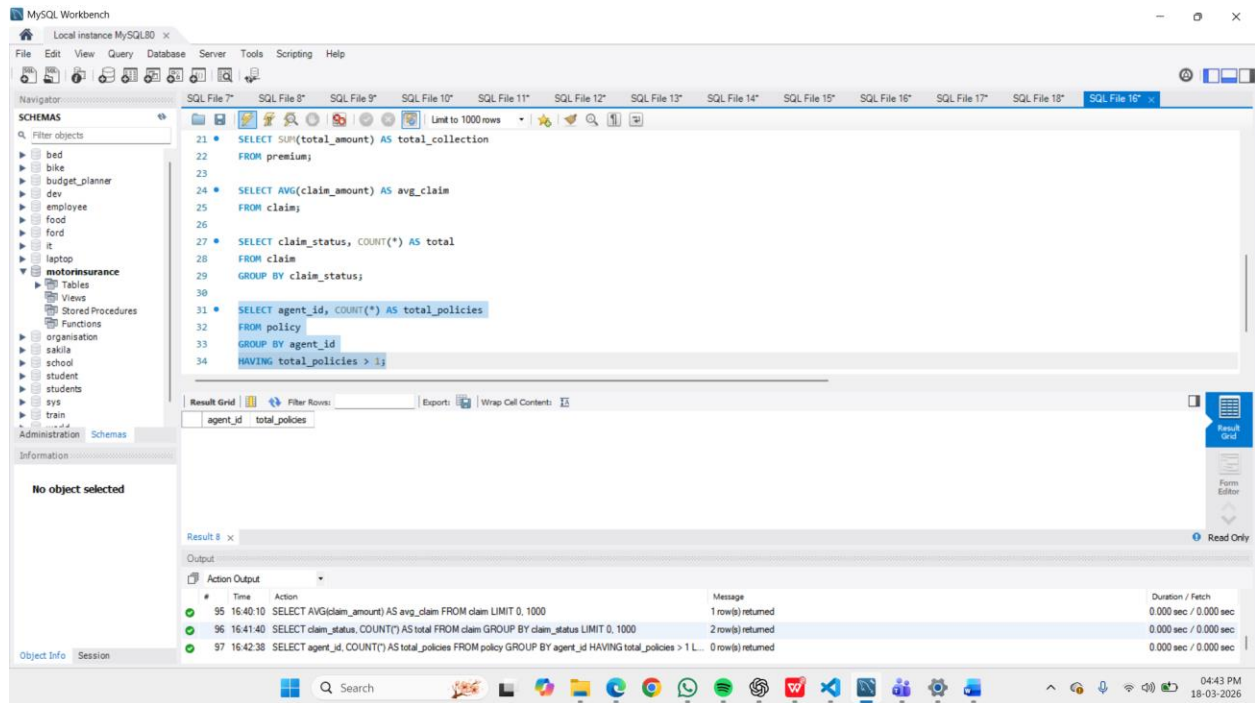
Claims Count by Status

```
SELECT claim_status, COUNT(*) AS total
FROM claim
GROUP BY claim_status;
```



HAVING Example

```
SELECT agent_id, COUNT(*) AS total_policies
FROM policy
GROUP BY agent_id
HAVING total_policies > 1;
```



6. Joins (Important for Project)

INNER JOIN

SELECT c.first_name, p.policy_id

FROM customers c

INNER JOIN trip_details td ON c.customer_id = td.customer_id

INNER JOIN policy p ON td.trip_id = p.trip_id;

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- bed
- bike
- budget_planner
- dev
- employee
- food
- ford
- it
- laptop
- motorinsurance
 - Tables
 - Views
 - Stored Procedures
 - Functions
- organisation
- sakila
- school
- student
- students
- sys
- train

Information

No object selected

Object Info Session

SQL File 7* SQL File 8* SQL File 9* SQL File 10* SQL File 11* SQL File 12* SQL File 13* SQL File 14* SQL File 15* SQL File 16* SQL File 17* SQL File 18*

Limit to 1000 rows

```

26
27 SELECT claim_status, COUNT(*) AS total
28 FROM claim
29 GROUP BY claim_status;
30
31 SELECT agent_id, COUNT(*) AS total_policies
32 FROM policy
33 GROUP BY agent_id
34 HAVING total_policies > 1;
35
36 SELECT c.first_name, p.policy_id
37 FROM customers c
38 INNER JOIN trip_details td ON c.customer_id = td.customer_id
39 INNER JOIN policy p ON td.trip_id = p.trip_id;

```

Result Grid

first_name	policy_id
Arun	1
Priya	2
Rahul	3
Sneha	4
Vijay	5
Anjali	6
Ravi	7

Output

Action Output

#	Time	Action	Message	Duration / Fetch
96	16:41:40	SELECT claim_status, COUNT(*) AS total FROM claim GROUP BY claim_status LIMIT 0. 1000	2 row(s) returned	0.000 sec / 0.000 sec
97	16:42:38	SELECT agent_id, COUNT(*) AS total_policies FROM policy GROUP BY agent_id HAVING total_policies > 1	0 row(s) returned	0.000 sec / 0.000 sec
98	16:44:01	SELECT c.first_name, p.policy_id FROM customers c INNER JOIN trip_details td ON c.customer_id = td.customer_id	25 row(s) returned	0.000 sec / 0.000 sec

04:44 PM
18-03-2026

LEFT JOIN

SELECT c.first_name, p.policy_id

FROM customers c

LEFT JOIN trip_details td ON c.customer_id = td.customer_id

LEFT JOIN policy p ON td.trip_id = p.trip_id;

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- bed
- bike
- budget_planner
- dev
- employee
- food
- ford
- it
- laptop
- motorinsurance
 - Tables
 - Views
 - Stored Procedures
 - Functions
- organisation
- sakila
- school
- student
- students
- sys
- train

Information

No object selected

Object Info Session

SQL File 7* SQL File 8* SQL File 9* SQL File 10* SQL File 11* SQL File 12* SQL File 13* SQL File 14* SQL File 15* SQL File 16* SQL File 17* SQL File 18*

Limit to 1000 rows

```

32 FROM policy
33 -- Execute the selected portion of the script or everything, if there is no selection
34 HAVING total_policies > 1;
35
36 SELECT c.first_name, p.policy_id
37 FROM customers c
38 INNER JOIN trip_details td ON c.customer_id = td.customer_id
39 INNER JOIN policy p ON td.trip_id = p.trip_id;
40
41 SELECT c.first_name, p.policy_id
42 FROM customers c
43 LEFT JOIN trip_details td ON c.customer_id = td.customer_id
44 LEFT JOIN policy p ON td.trip_id = p.trip_id;
45

```

Result Grid

first_name	policy_id
Arun	1
Priya	2
Rahul	3
Sneha	4
Vijay	5
Anjali	6
Ravi	7

Output

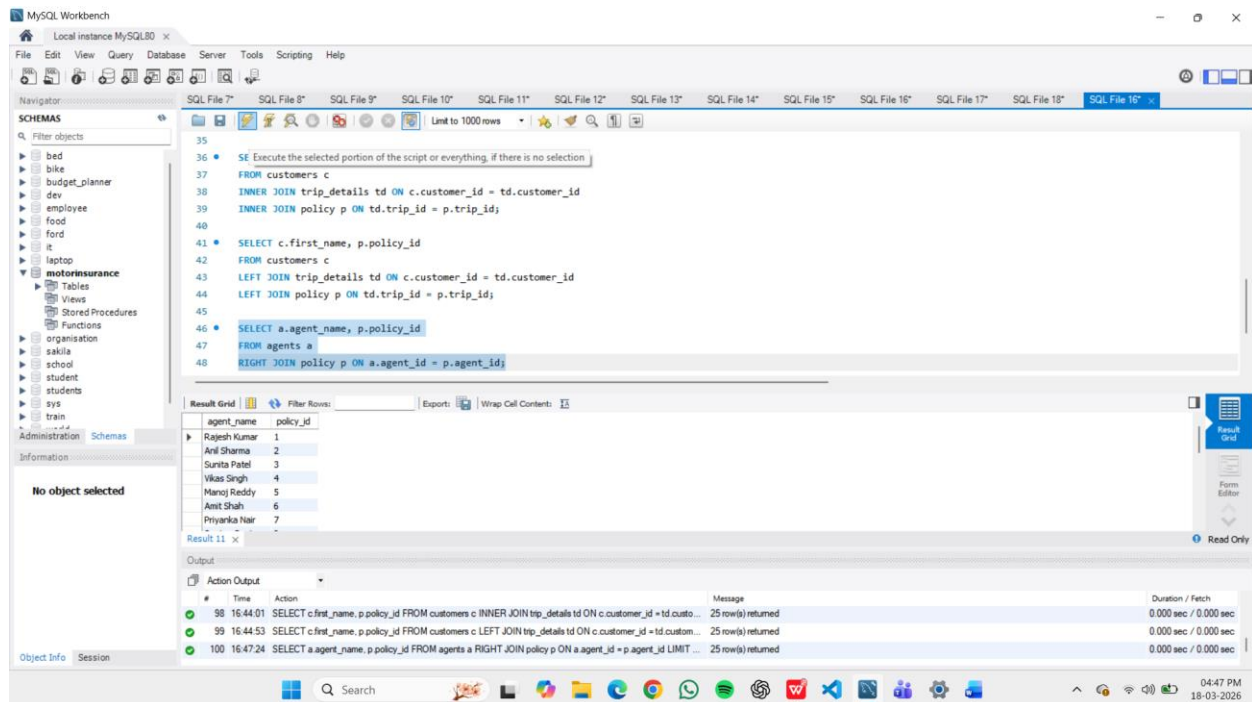
Action Output

#	Time	Action	Message	Duration / Fetch
97	16:42:38	SELECT agent_id, COUNT(*) AS total_policies FROM policy GROUP BY agent_id HAVING total_policies > 1	0 row(s) returned	0.000 sec / 0.000 sec
98	16:44:01	SELECT c.first_name, p.policy_id FROM customers c INNER JOIN trip_details td ON c.customer_id = td.customer_id	25 row(s) returned	0.000 sec / 0.000 sec
99	16:44:53	SELECT c.first_name, p.policy_id FROM customers c LEFT JOIN trip_details td ON c.customer_id = td.customer_id	25 row(s) returned	0.000 sec / 0.000 sec

04:44 PM
18-03-2026

RIGHT JOIN

```
SELECT a.agent_name, p.policy_id
FROM agents a
RIGHT JOIN policy p ON a.agent_id = p.agent_id;
```

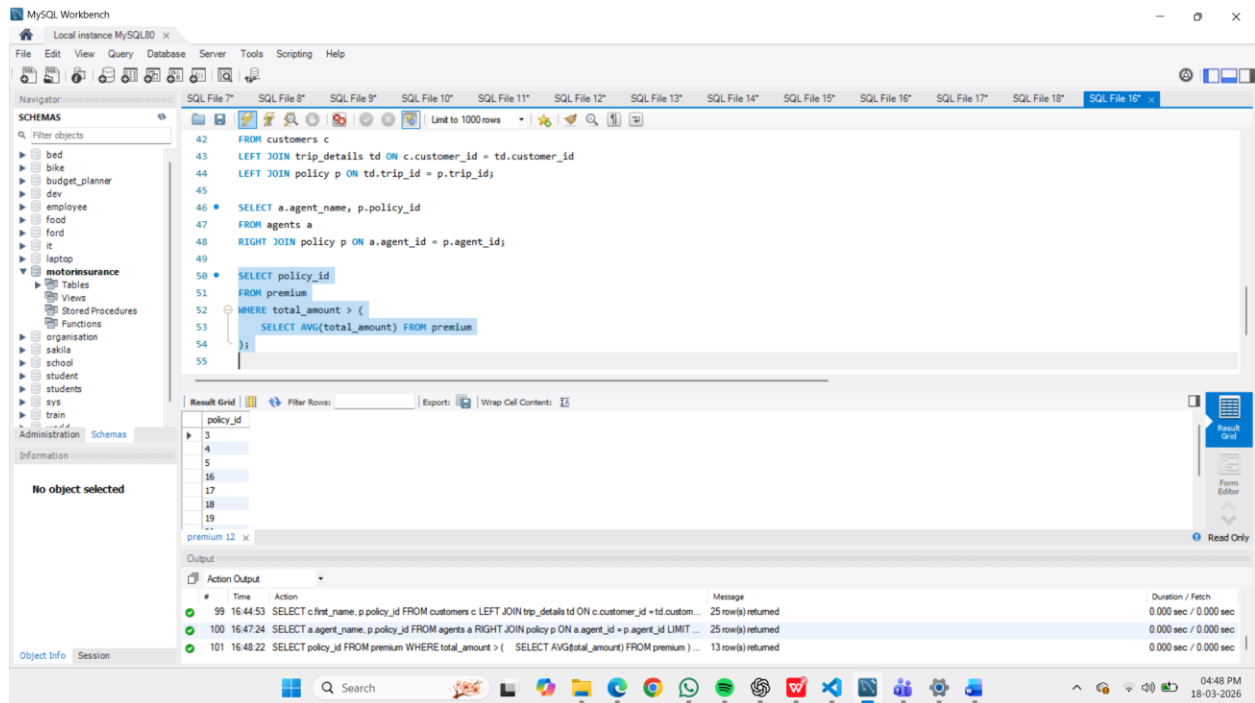


7. Subqueries

Subquery in WHERE

Policies with premium greater than average:

```
SELECT policy_id
FROM premium
WHERE total_amount > (
    SELECT AVG(total_amount) FROM premium
);
```

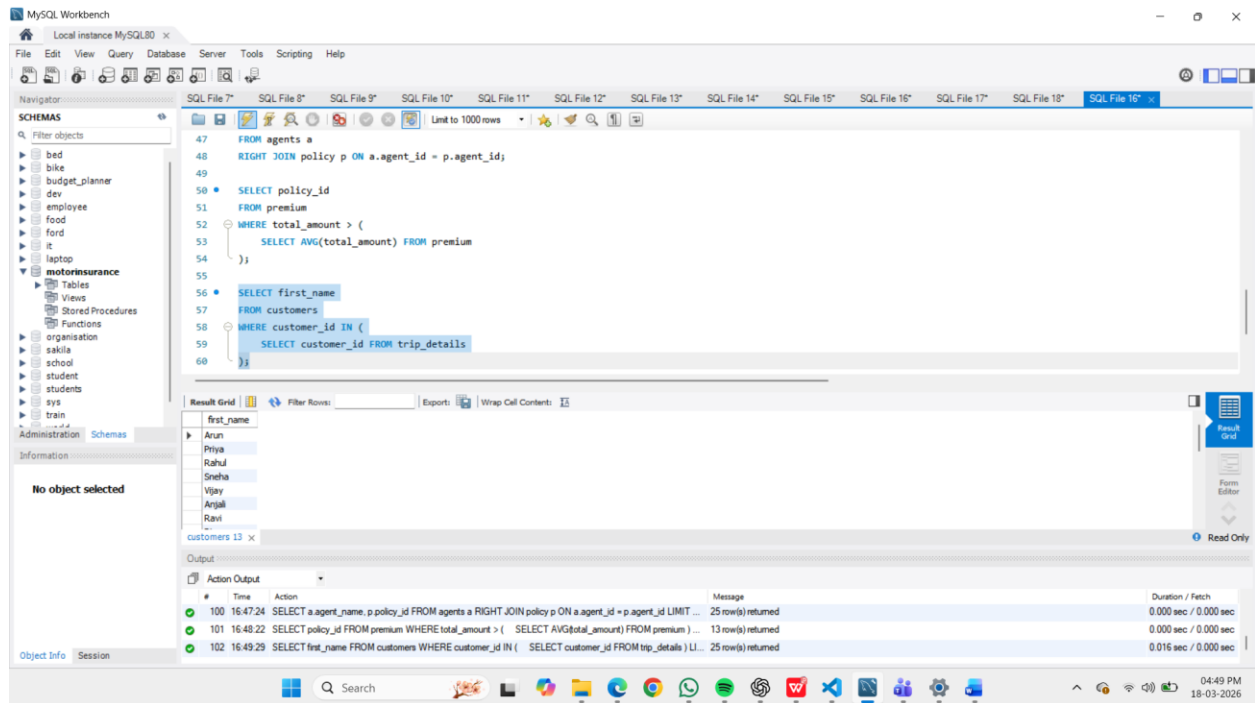


Subquery with IN

```

SELECT first_name
FROM customers
WHERE customer_id IN (
    SELECT customer_id FROM trip_details
);

```



8. Advanced Queries

CASE Statement

SELECT policy_id,

CASE

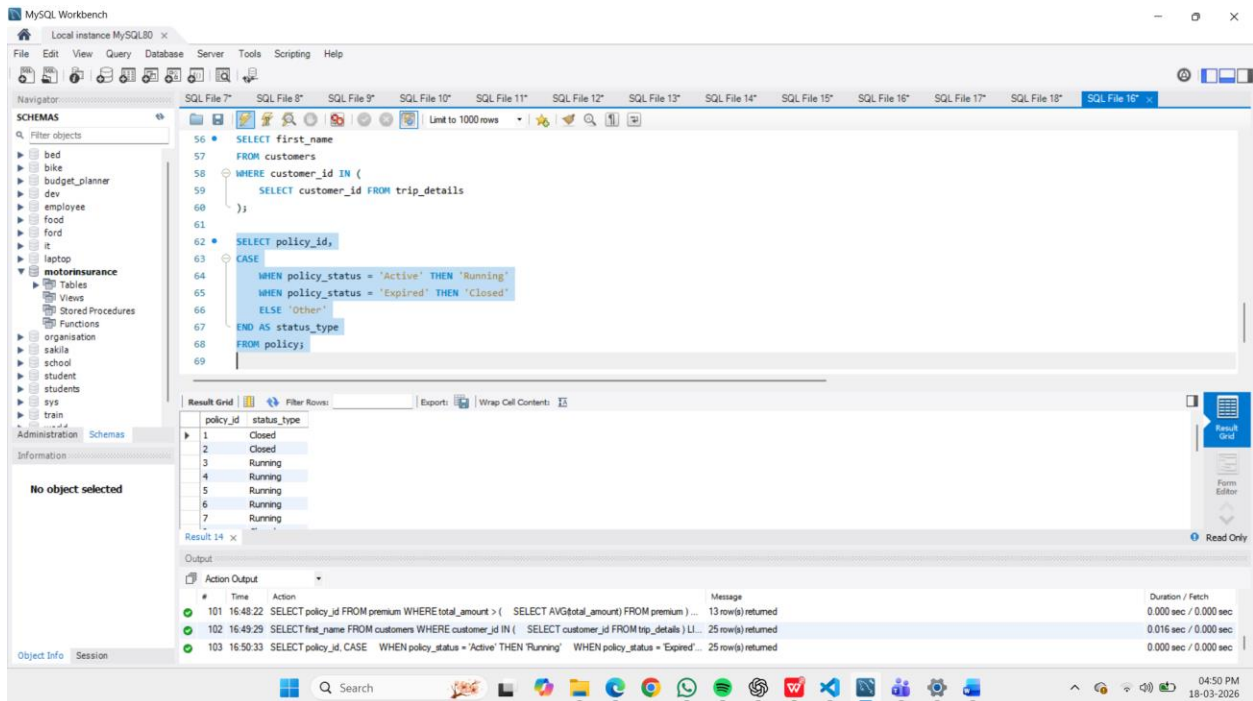
 WHEN policy_status = 'Active' THEN 'Running'

 WHEN policy_status = 'Expired' THEN 'Closed'

 ELSE 'Other'

END AS status_type

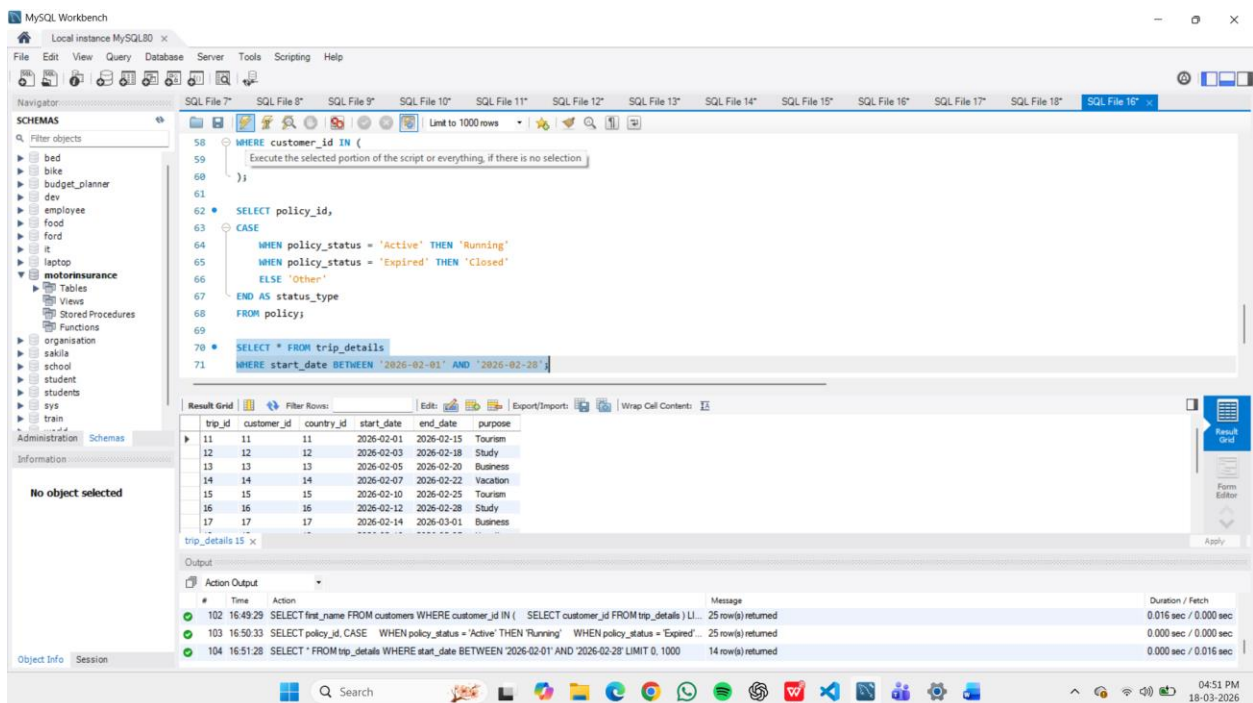
FROM policy;



BETWEEN

SELECT * FROM trip_details

WHERE start_date BETWEEN '2026-02-01' AND '2026-02-28';



LIKE

```
SELECT * FROM customers
WHERE first_name LIKE 'A%';
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
61
62 SELECT policy_id,
63 CASE
64     WHEN policy_status = 'Active' THEN 'Running'
65     WHEN policy_status = 'Expired' THEN 'Closed'
66     ELSE 'Other'
67 END AS status_type
68 FROM policy;
69
70 SELECT * FROM trip_details
71 WHERE start_date BETWEEN '2026-02-01' AND '2026-02-28';
72
73 SELECT * FROM customers
74 WHERE first_name LIKE 'A%';
```

The Results Grid shows the results of the last query (line 74):

customer_id	first_name	last_name	gender	dob	email	phone	address
1	Arun	Kumar	Male	1998-01-10	arun1@gmail.com	9000000001	Chennai
6	Anjali	Mehta	Female	1995-06-22	anjali6@gmail.com	9000000006	Mumbai
12	Ajay	Singh	Male	1989-12-14	ajay12@gmail.com	9000000012	Delhi
14	Arvind	Rao	Male	1992-02-18	arvind14@gmail.com	9000000014	Hyderabad
24	Arsha	Desai	Female	1995-12-30	arsha24@gmail.com	9000000024	Mumbai

The Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
103	16:50:33	SELECT policy_id, CASE WHEN policy_status = 'Active' THEN 'Running' WHEN policy_status = 'Expired' THEN 'Closed' ELSE 'Other' END AS status_type FROM policy;	25 row(s) returned	0.000 sec / 0.000 sec
104	16:51:28	SELECT * FROM trip_details WHERE start_date BETWEEN '2026-02-01' AND '2026-02-28' LIMIT 0, 1000	14 row(s) returned	0.000 sec / 0.016 sec
105	16:52:57	SELECT * FROM customers WHERE first_name LIKE 'A%' LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec

DISTINCT

```
SELECT DISTINCT region FROM agents;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
63 CASE
64     WHEN policy_status = 'Active' THEN 'Running'
65     WHEN policy_status = 'Expired' THEN 'Closed'
66     ELSE 'Other'
67 END AS status_type
68 FROM policy;
69
70 SELECT * FROM trip_details
71 WHERE start_date BETWEEN '2026-02-01' AND '2026-02-28';
72
73 SELECT * FROM customers
74 WHERE first_name LIKE 'A%';
75
76 SELECT DISTINCT region FROM agents;
```

The Results Grid shows the results of the last query (line 76):

region
South
North
West
East
Central

The Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
104	16:51:28	SELECT * FROM trip_details WHERE start_date BETWEEN '2026-02-01' AND '2026-02-28' LIMIT 0, 1000	14 row(s) returned	0.000 sec / 0.016 sec
105	16:52:57	SELECT * FROM customers WHERE first_name LIKE 'A%' LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
106	16:53:54	SELECT DISTINCT region FROM agents LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec

LIMIT & OFFSET

SELECT * FROM customers

LIMIT 5 OFFSET 5;

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following queries:

```
66 ELSE 'Other'
67 E) Execute the selected portion of the script or everything, if there is no selection
68 FROM policy;
69
70 SELECT * FROM trip_details
71 WHERE start_date BETWEEN '2026-02-01' AND '2026-02-28';
72
73 SELECT * FROM customers
74 WHERE first_name LIKE 'A%';
75
76 SELECT DISTINCT region FROM agents;
77
78 SELECT * FROM customers
79 LIMIT 5 OFFSET 5;
```

The Result Grid shows the results of the last query, displaying 5 rows of customer data:

customer_id	first_name	last_name	gender	dob	email	phone	address
6	Anjali	Mehta	Female	1995-06-22	anjali6@gmail.com	9000000006	Mumbai
7	Ravi	Patel	Male	1994-07-10	ravi7@gmail.com	9000000007	Ahmedabad
8	Divya	Nair	Female	1993-08-11	divya8@gmail.com	9000000008	Kochi
9	Suresh	Babu	Male	1992-09-12	suresh9@gmail.com	9000000009	Madurai
10	Kiran	Das	Male	1991-10-10	kiran10@gmail.com	9000000010	Kolkata

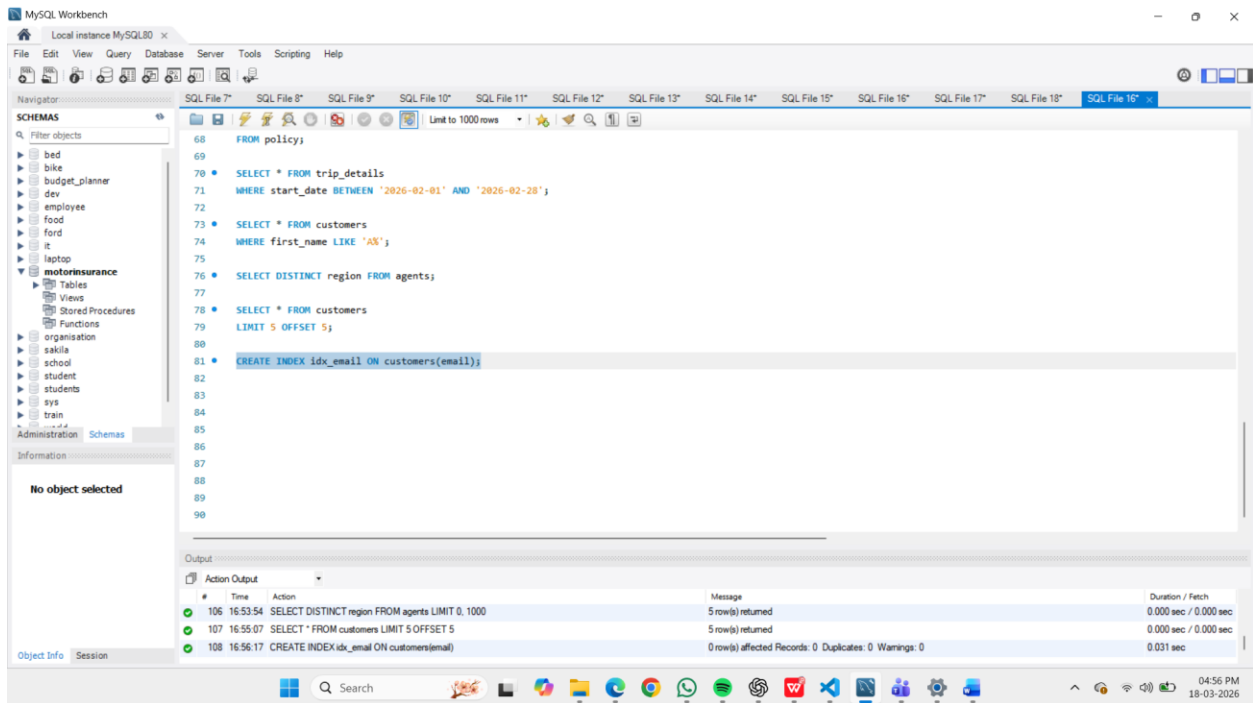
The Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
105	16:52:57	SELECT * FROM customers WHERE first_name LIKE 'A%'; LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
106	16:53:54	SELECT DISTINCT region FROM agents LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
107	16:55:07	SELECT * FROM customers LIMIT 5 OFFSET 5	5 row(s) returned	0.000 sec / 0.000 sec

9. Indexing

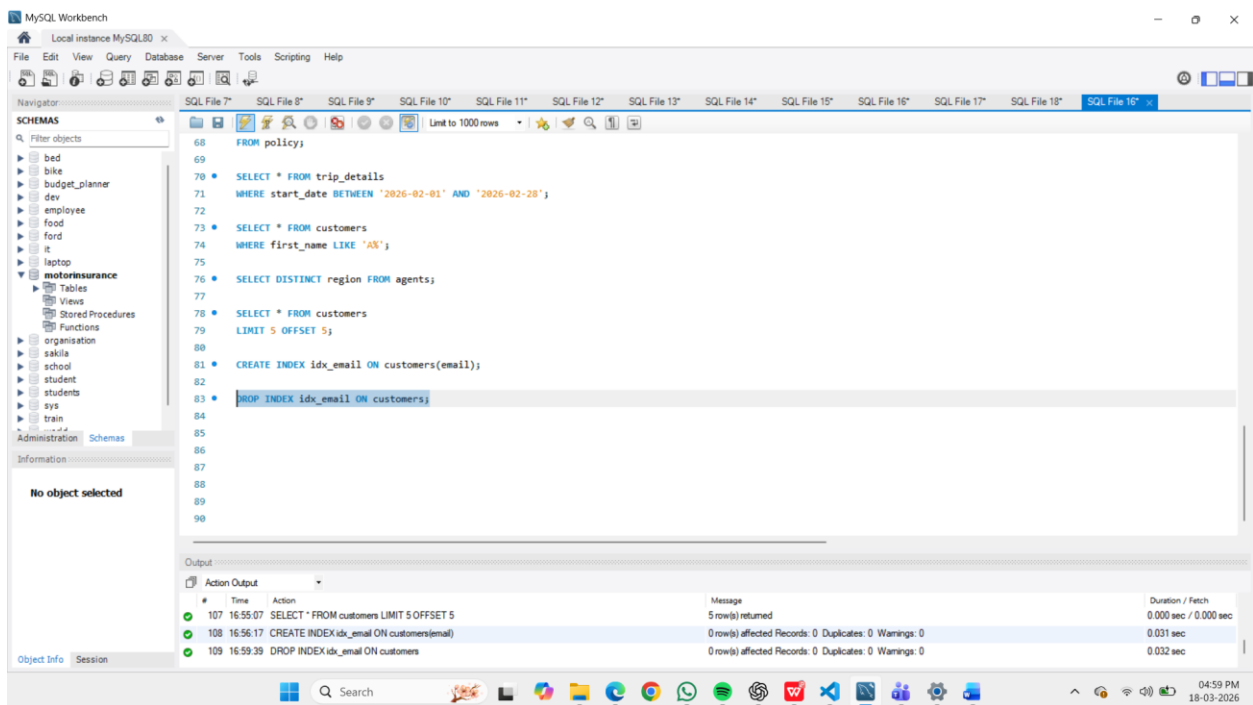
Create Index

CREATE INDEX idx_email ON customers(email);



Drop Index

DROP INDEX idx_email ON customers;



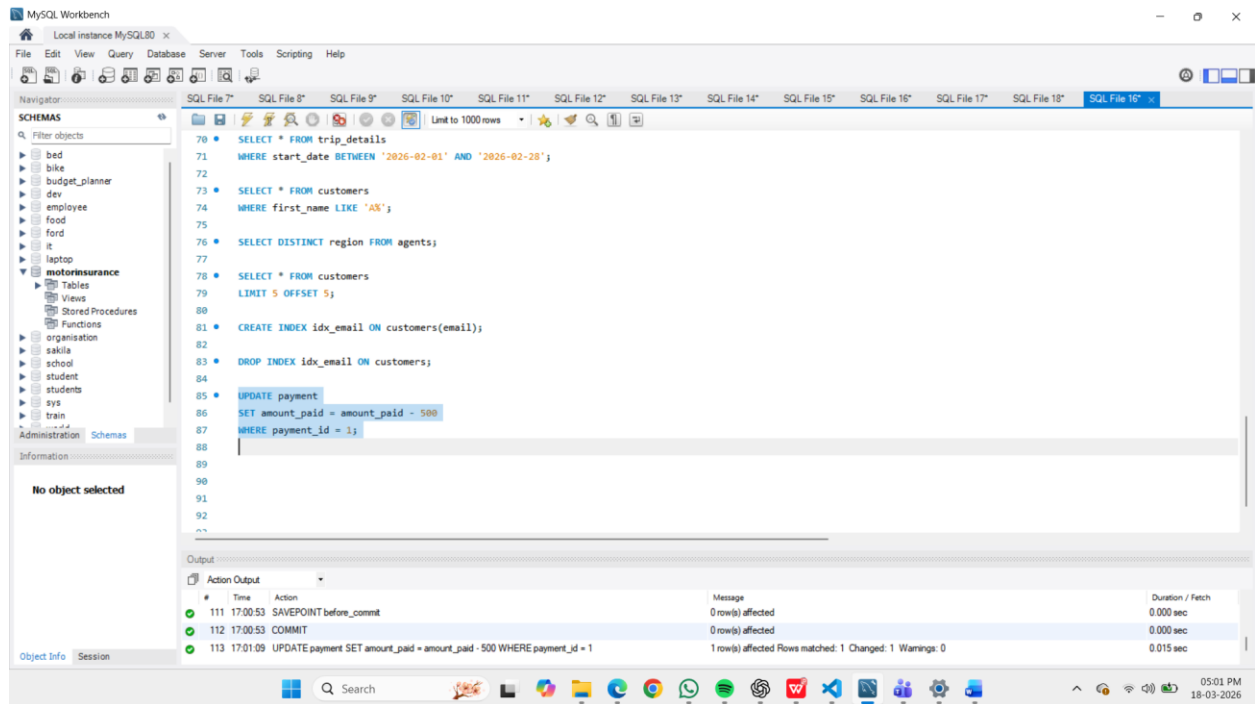
10. Transactions & ACID

START TRANSACTION;

UPDATE payment

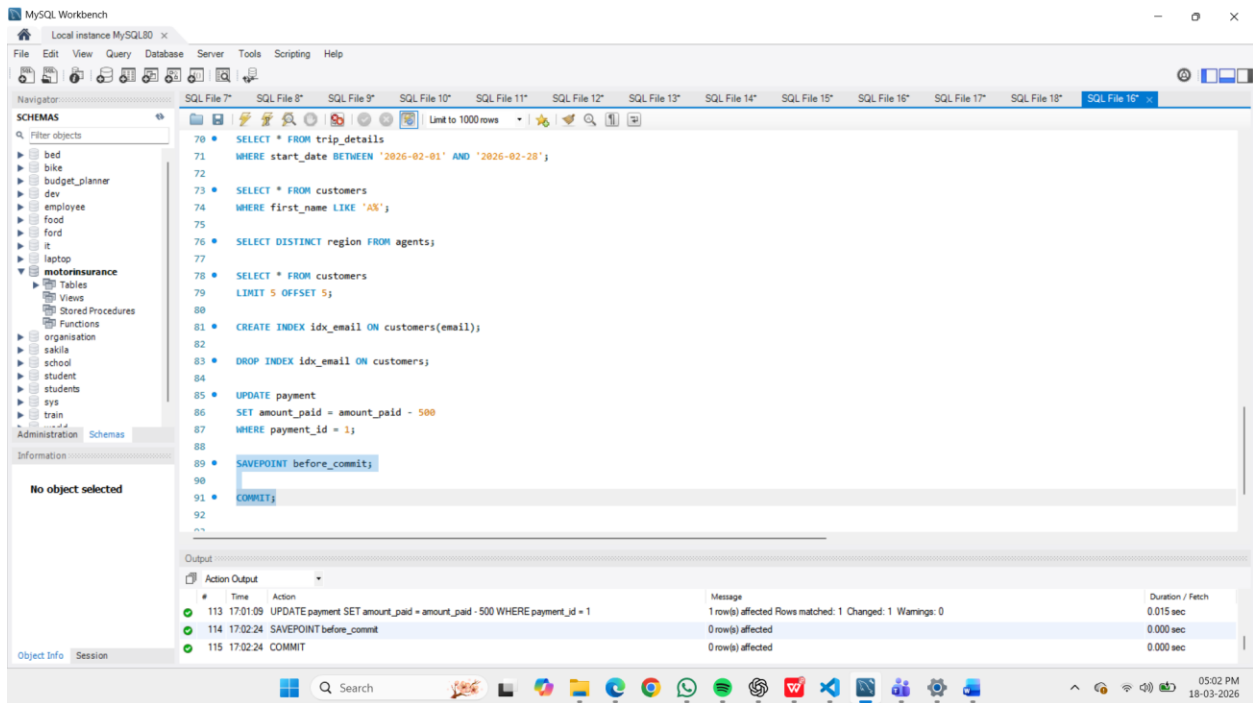
SET amount_paid = amount_paid - 500

WHERE payment_id = 1;



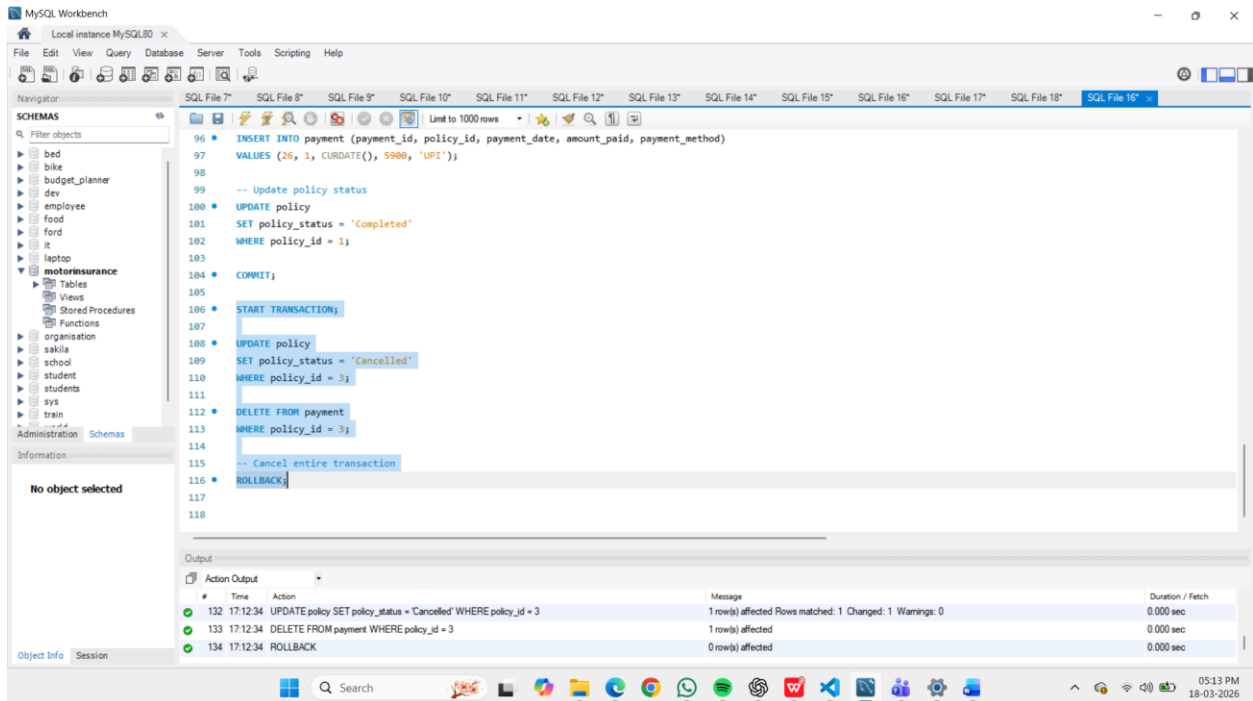
SAVEPOINT before_commit;

COMMIT;



Rollback example:

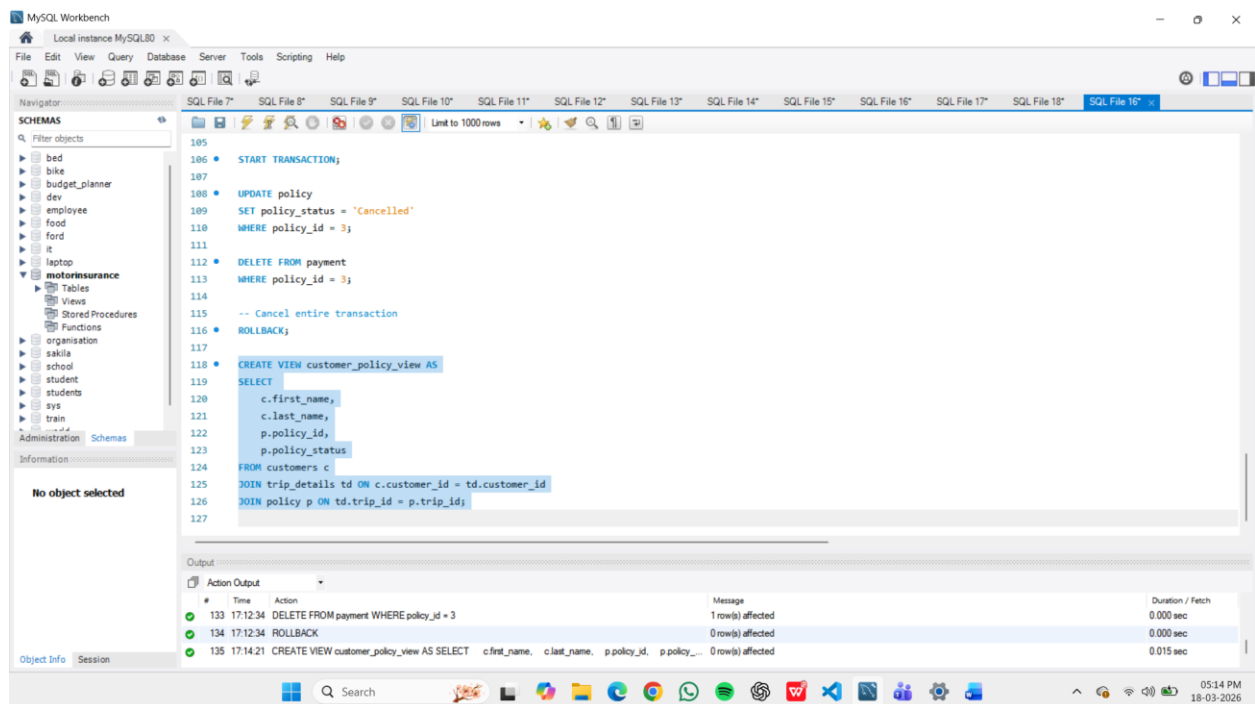
ROLLBACK TO before_commit;



11. Views

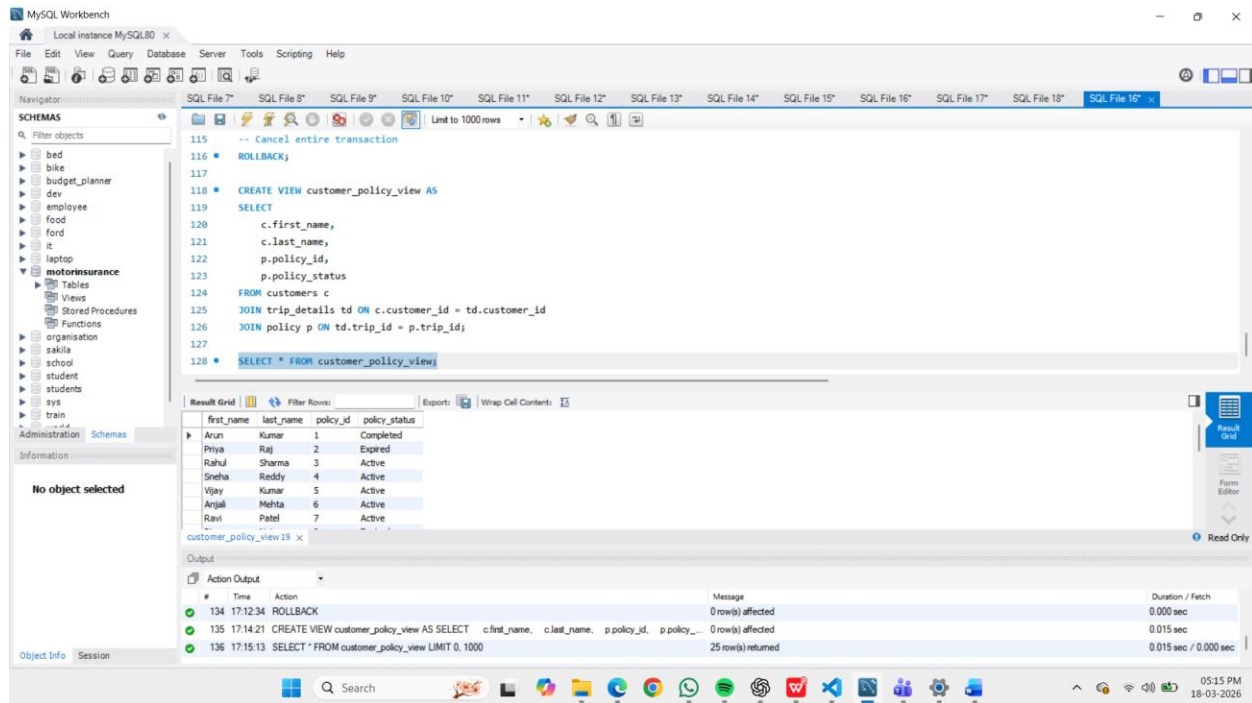
Create View

```
CREATE VIEW customer_policy_view AS
SELECT
    c.first_name,
    c.last_name,
    p.policy_id,
    p.policy_status
FROM customers c
JOIN trip_details td ON c.customer_id = td.customer_id
JOIN policy p ON td.trip_id = p.trip_id;
```



Use it:

```
SELECT * FROM customer_policy_view;
```



12. Stored Procedure

Procedure to Get Customer Policies

DELIMITER //

CREATE PROCEDURE GetCustomerPolicies(IN cust_id INT)

BEGIN

 SELECT p.policy_id, p.policy_status

 FROM policy p

 JOIN trip_details td ON p.trip_id = td.trip_id

 WHERE td.customer_id = cust_id;

END //

DELIMITER ;

Call it:

CALL GetCustomerPolicies(1);

13. Triggers

BEFORE INSERT Trigger on Claim

DELIMITER //

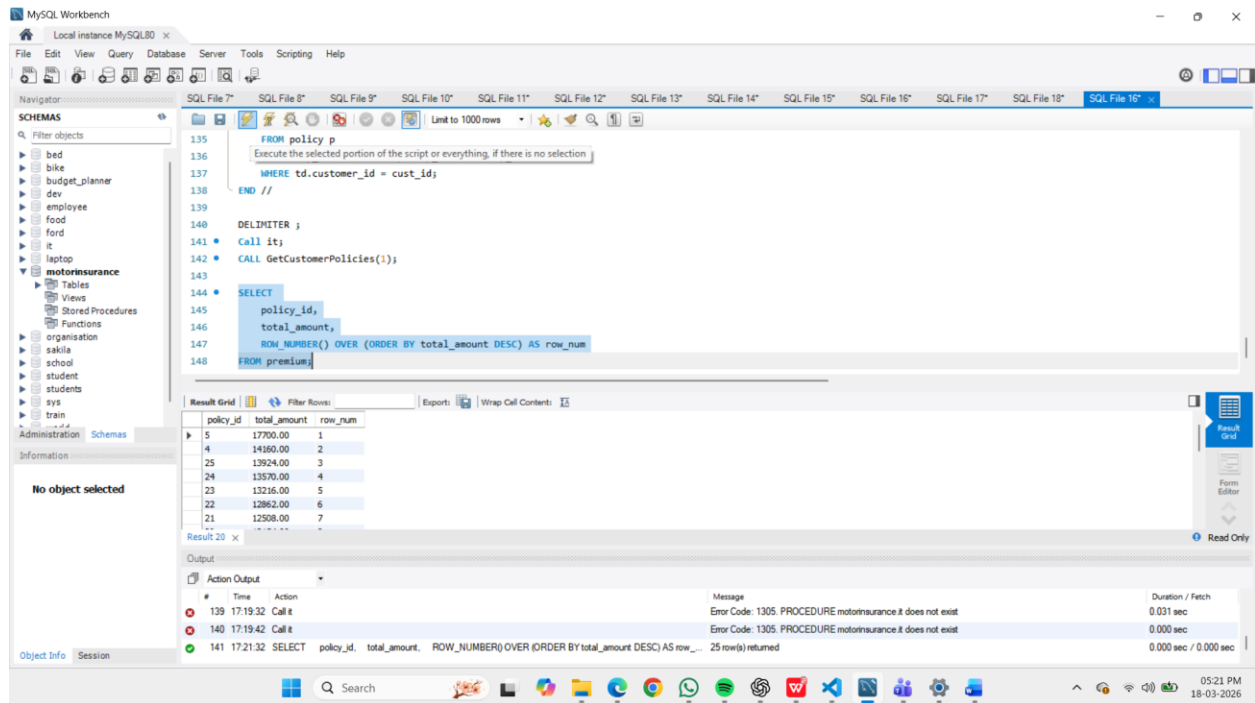
```
CREATE TRIGGER before_claim_insert
BEFORE INSERT ON claim
FOR EACH ROW
BEGIN
    IF NEW.claim_amount <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Claim amount must be greater than zero';
    END IF;
END //
```

DELIMITER ;

14. Window Functions

ROW_NUMBER

```
SELECT
    policy_id,
    total_amount,
    ROW_NUMBER() OVER (ORDER BY total_amount DESC) AS row_num
FROM premium;
```



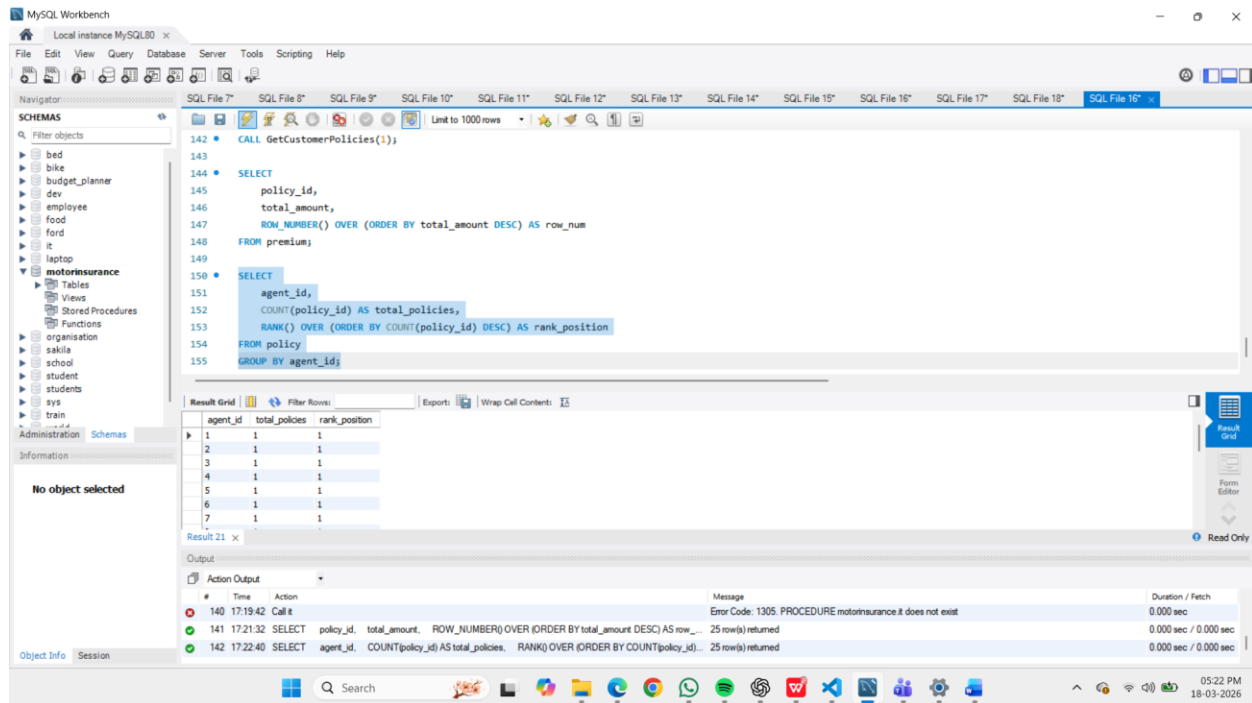
RANK

SELECT

```

    agent_id,
    COUNT(policy_id) AS total_policies,
    RANK() OVER (ORDER BY COUNT(policy_id) DESC) AS rank_position
FROM policy
GROUP BY agent_id;

```



15. Error Handling (Procedure Example)

DELIMITER //

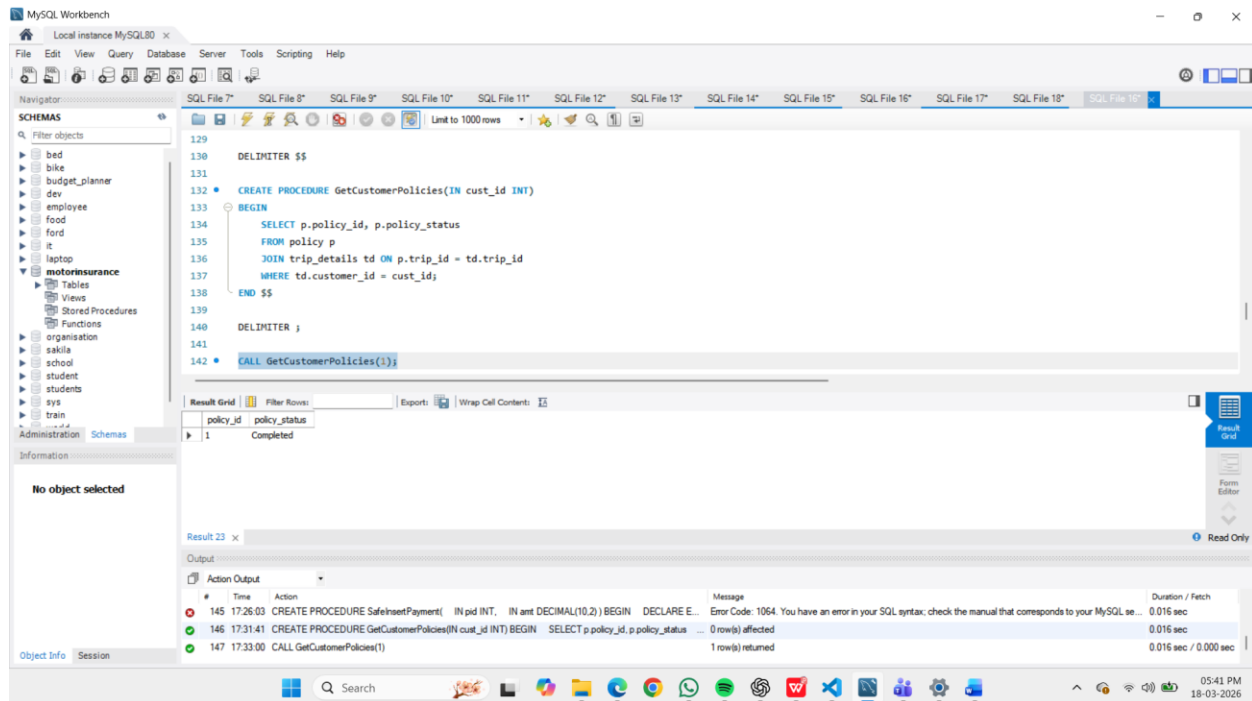
```

CREATE PROCEDURE SafeInsertPayment(
    IN pid INT,
    IN amt DECIMAL(10,2)
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
    END;

    START TRANSACTION;
    INSERT INTO payment VALUES (100, pid, CURDATE(), amt, 'UPI');
    COMMIT;
END //

```

DELIMITER ;

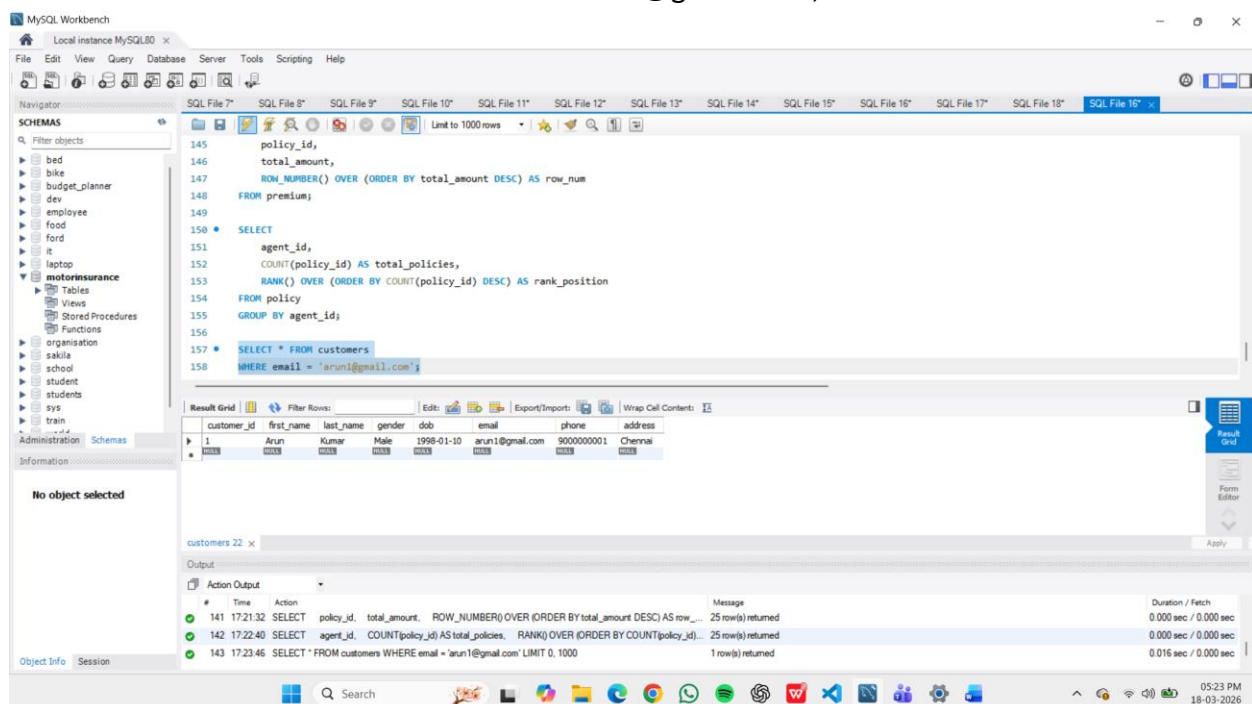


16. Performance Optimization Example

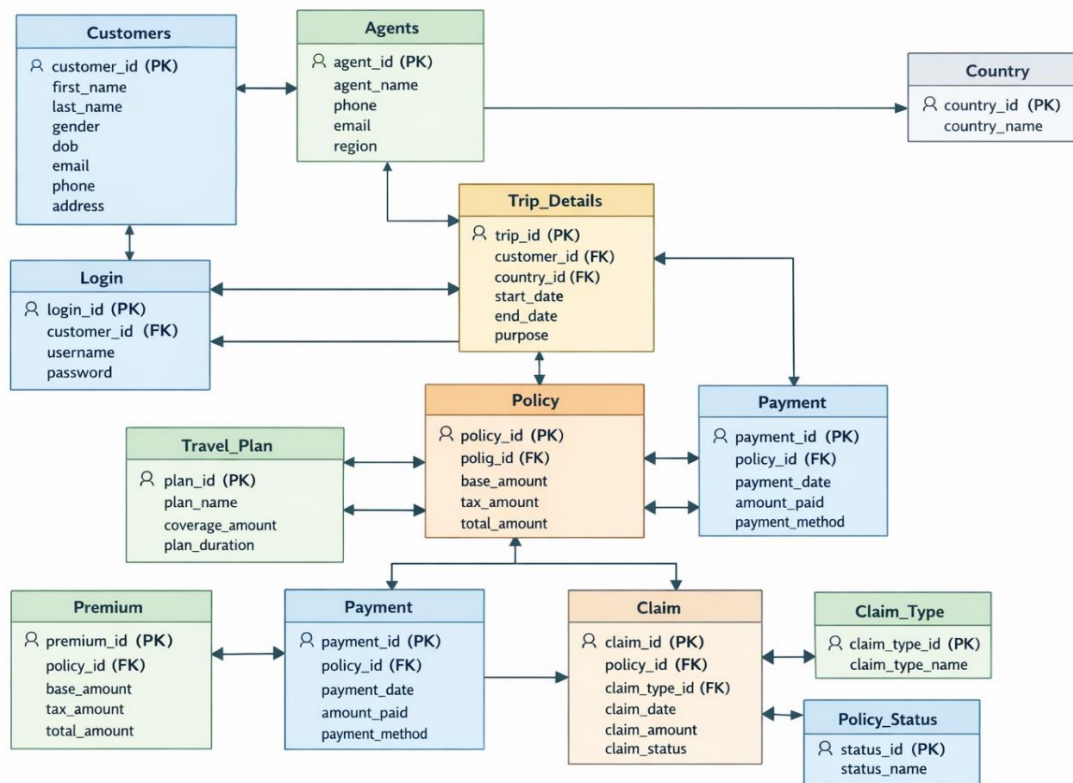
Check query plan:

EXPLAIN

SELECT * FROM customers WHERE email = 'arun1@gmail.com';



ER Diagram:



Conclusion:

The Travel Insurance Management System provides a structured and reliable solution for managing insurance operations using MySQL. It integrates customers, agents, travel plans, policies, payments, and claims into a well-connected relational database.

The system ensures data accuracy through primary and foreign keys, constraints, and normalization. SQL features such as joins, aggregation, subqueries, views, procedures, triggers, and transactions support efficient data processing and reporting.

Overall, the project demonstrates practical database design and implementation skills, offering a scalable foundation for real-world travel insurance management.

